Manual académico y práctico de investigación de operaciones con QSB

Guía de uso, modelación e interpretación de resultados

Proyecto QSB

Editores: comunidad académica y de desarrollo de QSB

Edición de trabajo verificada
11 de octubre de 2026

Nota editorial

Este manual acompaña la aplicación macOS y la herramienta de línea de comandos de QSB. Presenta fundamentos de investigación de operaciones, ejemplos originales reproducibles e instrucciones para formular, validar, resolver e interpretar modelos. La cobertura corresponde al estado documentado del proyecto el 11 de octubre de 2026; la hoja de ruta sigue siendo la referencia pública para el estado del software.

Los ejemplos de `manual/examples/` fueron creados para este proyecto. No necesitan archivos de WinQSB ni reproducen su manual. Las secciones de compatibilidad explican cómo importar archivos propios sin redistribuir materiales propietarios.

Cómo leer el manual

Empiece por la guía de la aplicación y el caso de programación lineal. Después consulte el capítulo de la familia que necesite. Cada capítulo distingue los supuestos del modelo, el procedimiento en QSB, los resultados y las limitaciones del método. Una solución numérica no certifica por sí sola que la formulación represente correctamente el problema real.

Los comandos se ejecutan desde la raíz del repositorio. Los números de JSON usan punto decimal; el texto matemático puede usar coma decimal. Los resultados se redondean para facilitar la lectura; las comparaciones de verificación usan tolerancias, no igualdad textual de números de coma flotante.

Reproducibilidad

Los archivos de entrada forman parte de las fuentes del manual. El verificador ejecuta los comandos asociados y comprueba los resultados esperados. La edición separa los solvers educativos nativos de los motores externos opcionales, y no atribuye garantías de optimalidad a heurísticas o métodos locales.

Índice general

Nota editorial II

I Uso de QSB y fundamentos 1

1. Guía práctica de la aplicación 2
1.1. Un primer recorrido reproducible . . . 2
1.2. Crear, importar y editar . . . 2
1.3. Validar, ejecutar e interpretar estados . . . 3
1.4. Guardar modelos y resultados . . . 3
1.5. Teclado y alternativas a los gráficos . . . 3
1.6. Terminal y JSON normalizado . . . 4
1.7. Diagnóstico de problemas . . . 5

2. Preliminares metodológicos 6
2.1. Objeto del manual . . . 6
2.2. Ciclo de modelación . . . 6
2.3. Validez y exactitud . . . 6
2.4. Trazabilidad computacional . . . 7
2.5. Comparación y sensibilidad . . . 7

II Optimización y decisiones 8

3. Programación lineal 9
3.1. Qué representa un modelo lineal . . . 9
3.2. Ejemplo: mezcla de producción . . . 9
3.2.1. Entrada y ejecución reproducibles . . . 10
3.2.2. Resultado e interpretación . . . 10
3.3. Dualidad y sensibilidad: qué puede deducirse . . . 10
3.4. Método, backends y límites . . . 11
3.5. Ejercicios . . . 11

4. Programación entera y mixta 12
4.1. Decisiones indivisibles . . . 12
4.2. Ejemplo: cobertura mínima con actividades enteras . . . 12
4.2.1. Ejecución en CLI y macOS . . . 12
4.2.2. Óptimo y comprobación . . . 13
4.3. Ramificación y acotación . . . 13
4.4. Interpretación y errores frecuentes . . . 14
4.5. Ejercicios . . . 14

5. Metas, programación cuadrática y no lineal **15**
5.1. Programación por metas . . . 15
5.1.1. Prioridades y desviaciones . . . 15
5.1.2. Ejemplo: proteger un compromiso de producción . . . 15
5.1.3. Flujo en macOS y carácter del solver . . . 16
5.2. Programación cuadrática . . . 16
5.2.1. Convención del objetivo y supuestos . . . 16
5.2.2. Ejemplo: ajuste con capacidad limitada . . . 17
5.2.3. Flujo en macOS y límites . . . 17
5.3. Programación no lineal . . . 18
5.3.1. Expresiones y dominio . . . 18
5.3.2. Ejemplo: distancia con restricción . . . 18
5.3.3. Uso en macOS y garantía aproximada . . . 19
5.4. Cómo comparar y revisar resultados . . . 19
5.5. Ejercicios . . . 19

6. Modelos de redes **21**
6.1. De una situación a un grafo . . . 21
6.2. Ruta más corta y árbol de expansión mínima . . . 21
6.3. Flujo máximo . . . 22
6.4. Vendedor viajero . . . 22
6.5. Asignación y transporte . . . 22
6.6. Flujo de costo mínimo . . . 23
6.7. Uso en macOS y elección del backend . . . 23
6.8. Ejercicios . . . 23

7. Decisiones, juegos y programación dinámica **25**
7.1. Elegir bajo incertidumbre . . . 25
7.1.1. Tabla de pagos e información . . . 25
7.1.2. Actualización bayesiana . . . 26
7.1.3. Árboles y análisis hacia atrás . . . 26
7.2. Juegos de suma cero . . . 27
7.3. Programación dinámica: estado, etapa y acción . . . 27
7.3.1. Mochila acotada . . . 28
7.3.2. Ruta por etapas (Stagecoach) . . . 28
7.3.3. Producción e inventario . . . 28
7.4. Uso, metadatos y límites comunes . . . 29
7.5. Ejercicios y comprobaciones . . . 29

III Planificación de operaciones **30**

8. Modelos de inventarios **31**
8.1. Qué decisión se modela . . . 31
8.2. EOQ y producción gradual . . . 31
8.2.1. Ejemplo: reposición de cuadernos . . . 31
8.3. Descuentos sobre todas las unidades . . . 32
8.4. Vendedor de periódicos . . . 32
8.5. Dimensionamiento de lotes . . . 32
8.6. Revisión con demanda incierta . . . 32
8.7. Procedimiento en macOS y trazabilidad . . . 33

8.8. Ejercicios . . . 33

9. Calendarización y secuenciación 34
9.1. Problema y supuestos . . . 34
9.2. Flow shop: una ruta común . . . 34
9.3. Job shop: rutas propias . . . 34
9.4. Leer el Gantt y usar macOS . . . 35
9.5. Ejercicios . . . 35

10. Proyectos y planificación de producción 36
10.1. CPM: precedencias y camino crítico . . . 36
10.2. PERT: duración esperada e incertidumbre . . . 37
10.2.1. CPM y PERT en macOS . . . 37
10.3. Planificación agregada . . . 37
10.4. MRP: explosión de necesidades . . . 38
10.5. Ejercicios . . . 39

11. Instalaciones y equilibrado de líneas 40
11.1. Localización de una instalación . . . 40
11.2. Equilibrado de una línea . . . 40
11.3. Distribución en planta . . . 41
11.4. Backend, validación y decisión . . . 42
11.5. Ejercicios . . . 42

IV Incertidumbre y calidad 43

12. Modelos de colas 44
12.1. Del flujo a la espera . . . 44
12.2. M/M/1 en régimen estacionario . . . 44
12.2.1. Ejemplo: una ventanilla . . . 44
12.3. Varios servidores y capacidad finita . . . 44
12.3.1. Ejemplo: dos ventanillas y tres plazas . . . 45
12.4. Costos, interfaz y límites . . . 45
12.5. Ejercicios . . . 46

13. Pronósticos y regresión 47
13.1. Una previsión es una hipótesis operativa . . . 47
13.2. Datos y errores . . . 47
13.3. Tendencia lineal: ejemplo completo . . . 47
13.4. Media móvil y suavizamiento . . . 48
13.5. Estacionalidad multiplicativa . . . 48
13.6. Regresión explicativa . . . 48
13.7. Trabajo en macOS y evaluación . . . 49
13.8. Ejercicios . . . 49

14. Procesos de Markov y simulación 50
14.1. Dos formas de describir la incertidumbre . . . 50
14.2. Cadena finita y convención de filas . . . 50
14.2.1. Ejemplo: disponibilidad de una máquina . . . 50
14.3. Simulación de eventos discretos . . . 51
14.3.1. Ejemplo: una ventanilla simulada . . . 51

14.4. Procedimiento en macOS 52
14.5. Ejercicios 52

15. Control de calidad y muestreo de aceptación 53
15.1. Vigilar un proceso o aceptar un lote 53
15.2. Cartas c y p 53
15.3. Carta de medias y rangos 54
15.4. Pareto y probabilidad normal 54
15.5. Señales e interfaz de calidad 54
15.6. Plan simple de aceptación 55
15.7. Plan doble y diseño 55
15.8. Ejercicios 56

A. Referencia de términos 57

Índice de tablas

3.1. Datos de la mezcla de producción 9
3.2. Óptimo lineal verificado con QSB 10

5.1. Resultado lexicográfico del caso de producción 16

Parte I

Uso de QSB y fundamentos

Capítulo 1

Guía práctica de la aplicación

1.1. Un primer recorrido reproducible

QSB ofrece una aplicación macOS y una herramienta de terminal llamada `qsb`. Ambas comparten los modelos, validadores y solvers de `QSBCore`. La interfaz actual usa etiquetas en inglés; este manual conserva esas etiquetas para que puedan localizarse en pantalla.

Para comenzar, abra `manual/examples/lp_sample.json` mediante **Model >Open Model...**. El modelo maximiza $50X_1 + 60X_2$ con $2X_1 + 3X_2 \leq 180$, $3X_1 + 2X_2 \leq 150$ y variables no negativas.

1. En **Definition**, compruebe nombres, coeficientes, relaciones, límites y tipos de variables. Aquí las variables son continuas.
2. Pulse **Validate** y revise **Validation**. Un modelo válido es una entrada coherente; todavía no es una solución.
3. En **Run**, seleccione **QSB Native** y ejecute el botón **Run**; también puede usar **Solve** en la barra de herramientas o **Solve >Run Current Model** en el menú.
4. En **Summary**, compruebe $X_1 = 18$, $X_2 = 48$ y objetivo 3780. Ambas restricciones se cumplen con igualdad. Revise también el estado de resolución y los metadatos antes de interpretar el número como óptimo.
5. Exporte el modelo y la solución por separado. Vuelva a abrir el modelo exportado para conservar una entrada reproducible.

El menú **Samples** permite explorar ejemplos internos, pero estos pueden ser distintos de los archivos del manual. Para reproducir un resultado citado, use el archivo indicado en su capítulo.

1.2. Crear, importar y editar

New Model... abre el selector de familias y crea un modelo inicial. **Open Model...** acepta JSON normalizado y formatos heredados reconocidos. La familia determina los controles que aparecen. Los editores nativos permiten trabajar con nombres, tablas y campos tipados; otras variantes usan un formulario estructurado y el JSON como superficie de intercambio.

El panel **Definition** define el problema; **Overview** resume su familia y estado. **JSON** permite inspeccionar la representación. Cuando se ofrece **Apply JSON to ... Editor**, aplique explícitamente el texto editado para reemplazar el borrador nativo. Escribir JSON no significa que sus cambios ya estén incorporados a la tabla. Revise los diagnósticos si falla la aplicación y conserve el texto que necesita reparar.

Algunas tablas ofrecen **Copy Table** y **Paste Table** para intercambiar rangos con hojas de cálculo. Revise encabezados, orden de columnas, separadores, unidades y nombres tras pegar. Los campos numéricos malformados no se convierten silenciosamente en ceros. En los editores que señalan dimensiones importadas incompletas, revise las celdas y los valores mostrados antes de aceptar **Use Displayed Dimensions**.

1.3. Validar, ejecutar e interpretar estados

La aplicación distingue modelo vacío, en edición, en validación, inválido y válido. La ejecución tiene estados independientes: **Not run**, **Validating**, **Solving**, **Solved** y **Failed**. Un estado Solved debe leerse junto a la condición matemática que informa la familia: un método puede devolver una aproximación, una solución local o una política heurística en lugar de un óptimo global.

Validation muestra errores y advertencias del modelo. Corrija los errores antes de resolver; una advertencia exige juicio, aunque permita seguir. **Run Details** permite revisar el backend y la información de ejecución. Cuando una ejecución larga muestra progreso, espere a que termine antes de repetir Run. No hay un control universal de cancelación; las rutas asíncronas de algunos editores evitan que una respuesta antigua sustituya un borrador nuevo. Esto no equivale a cancelar el cálculo en segundo plano.

QSB Native Métodos educativos deterministas. La exactitud y los límites dependen de la familia y se explican en sus capítulos.

Validate only Comprueba la entrada y produce diagnósticos; no calcula decisiones ni valores óptimos.

External solver Integraciones disponibles sólo para determinadas familias. La disponibilidad depende de la configuración y del solver instalado. No interprete la presencia de la opción como soporte externo universal.

Para LP/ILP y QP, consulte las condiciones de HiGHS en el capítulo de optimización. Las demás familias pueden indicar que el backend externo no está disponible. Cambie a QSB Native o Validate only según el trabajo que quiera hacer.

1.4. Guardar modelos y resultados

El botón **Export** de la barra prepara el JSON a partir del borrador actual. Si un campo nativo no puede convertirse al modelo, QSB muestra validación en vez de exportar datos antiguos. El menú **Model >Export Model JSON...** abre la exportación del texto de modelo; para asegurar que los campos recientes estén sincronizados, use primero Validate o el botón Export de la barra. **Export Solution JSON...** guarda el resultado existente y se habilita cuando hay contenido de solución.

Son documentos distintos: el modelo describe qué problema resolver y la solución registra qué devolvió una ejecución. Los formatos de salida varían entre familias; algunos incluyen el modelo y otros no. Conserve siempre ambos, además del comando, backend, versión del software y supuestos de su estudio. No use un documento de solución como si fuera una entrada de modelo.

1.5. Teclado y alternativas a los gráficos

Los atajos definidos por la aplicación son:

Acción	Teclas
New Model	Cmd+N
Open Model	Cmd+O
Export Model JSON	Cmd+S
Export Solution JSON	Cmd+Shift+S
Validate Model	Cmd+Shift+V
Run Current Model	Cmd+R
Solve ILP	Cmd+I

Los botones se habilitan según la familia y el estado. Solve ILP y Solve LP Relaxation son acciones específicas de programación lineal/entera; la relajación no impone integralidad. Algunos editores usan Cmd+Shift+N para añadir filas. Use Tab para desplazarse por controles nativos y los menús para localizar las acciones; el detalle del foco depende del editor.

Las tablas y el JSON permiten consultar los datos sin depender del color de un gráfico. Los resultados nativos incorporan etiquetas y valores accesibles en varias familias, pero esto no acredita por sí solo una revisión completa con VoiceOver. Para un análisis reproducible, conserve las tablas numéricas. La ventana macOS tiene mínimo de 980 por 620 puntos; use las áreas desplazables y cierre el Inspector cuando necesite más anchura.

1.6. Terminal y JSON normalizado

Desde la raíz del repositorio, SwiftPM construye y ejecuta la herramienta:

```
swift run qsb
swift run qsb validate manual/examples/lp_sample.json
swift run qsb solve-json manual/examples/lp_sample.json --backend native
```

Sin argumentos, `qsb` muestra el catálogo de comandos. Tras `swift build`, puede ejecutar `.build/debug/qsb` directamente. Para guardar la salida y separar errores:

```
.build/debug/qsb solve-json manual/examples/lp_sample.json \
  --backend native > /tmp/qsb-lp-solution.json 2> /tmp/qsb-lp-errors.txt
```

El JSON usa punto decimal, cadenas entre comillas dobles y `null` para los campos opcionales que lo admiten. No incluye comentarios. Cada familia posee un esquema propio: LP usa un objeto directo, mientras muchas otras usan un envoltorio con `kind` y `model`. Copie la estructura del ejemplo de la familia, manteniendo alineados nombres, vectores y matrices.

`solve-json` detecta la familia de una entrada normalizada. `validate` acepta entradas normalizadas o heredadas. `solve` está dirigido a modelos heredados; no sustituya indiscriminadamente un comando por otro. Los capítulos ofrecen los comandos específicos de cada familia, como `solve-dp-json` o `solve-decision-json`. `validate-json` es específico de LP/ILP, no un validador universal.

Para importar una entrada heredada propia, la ruta general es:

```
qsb import-legacy-json /ruta/al/modelo > /tmp/qsb-model.json
qsb validate /tmp/qsb-model.json
qsb solve-json /tmp/qsb-model.json --backend native
```

Aquí `qsb` presupone que el ejecutable está en PATH; puede sustituirlo por `swift run qsb`. Los archivos originales deben ser accesibles localmente y usados conforme a sus permisos. Los ejemplos del manual son independientes de materiales propietarios.

1.7. Diagnóstico de problemas

JSON ilegible Revise sintaxis, nombres de claves y tipos; un error de decodificación puede ocurrir antes de generar diagnósticos estructurados.

Modelo inválido Consulte `isValid`, gravedad, código, mensaje y ruta del diagnóstico cuando la salida los incluya. Corrija la entrada y vuelva a validar.

Sin solución factible La estructura puede ser válida y, aun así, impedirse satisfacer las restricciones o demandas. Revise límites y balances.

Backend no disponible Compruebe qué familias lo soportan y la configuración. Validate only no proporciona un resultado alternativo de solver.

Resultado inesperado Revise unidades, signo del objetivo, orden de estados, restricciones, opciones y exactitud. Compare con cálculos sencillos antes de cambiar de algoritmo.

La CLI termina con éxito cuando completa el comando y con fallo ante errores lanzados, incluido uso incorrecto. Algunas validaciones retornan normalmente un informe con `isValid: false`; el código de salida por sí solo no prueba que el modelo sea válido. Un flujo automático debe revisar también el documento de validación y el estado del resultado. La ausencia de errores de sintaxis no demuestra que el modelo represente bien el sistema real.

Capítulo 2

Preliminares metodológicos

2.1. Objeto del manual

La investigación de operaciones construye y analiza modelos cuantitativos para apoyar decisiones bajo restricciones. Este manual combina fundamentos y uso práctico: cada ejemplo transforma un enunciado en datos de QSB y explica qué significa el resultado. Los problemas son didácticos originales, con entradas normalizadas disponibles en `manual/examples/`; no requieren manuales ni archivos propietarios de programas anteriores.

Puede empezar por la guía de la aplicación y un ejemplo de programación lineal, o ir al capítulo de la familia que corresponde a su problema. Antes de ejecutar, lea sus supuestos: dos modelos que usan las mismas palabras pueden representar sistemas distintos. No confunda inventarios deterministas con demanda aleatoria, una ruta acíclica con una red general ni un óptimo local con uno global.

2.2. Ciclo de modelación

Un estudio riguroso registra:

1. el sistema real, el horizonte y la decisión que se desea apoyar;
2. variables controlables, parámetros y unidades;
3. supuestos, incertidumbre y alcance de los datos;
4. objetivo, restricciones y condiciones iniciales;
5. método de solución, backend y opciones de ejecución;
6. comprobaciones de factibilidad e interpretación;
7. sensibilidad, riesgos y límites de aplicación.

Escriba cada parámetro con su unidad. Si la demanda se expresa por año y el plazo en días, convierta a una base común antes de calcular. Un número sin unidad puede producir una solución matemáticamente coherente y operativamente incorrecta.

2.3. Validez y exactitud

La validación comprueba estructura, dimensiones, campos y reglas de la familia. No garantiza factibilidad ni que los supuestos sean adecuados. Un solver puede probar optimalidad

para un modelo, devolver una heurística o detenerse dentro de un límite. Consulte estado, algoritmo, exactitud y notas de ejecución.

En los metadatos, `exact` caracteriza el procedimiento para el problema admitido; no implica aritmética simbólica ni precisión infinita. `fixtureScale` señala un método educativo orientado a casos pequeños. La resolución numérica usa tolerancias; compare resultados con un margen coherente con la escala en lugar de exigir igualdad textual de decimales. La repetibilidad tampoco prueba validez: un error de formulación puede ser perfectamente determinista.

2.4. Trazabilidad computacional

Para reproducir un análisis, conserve entrada JSON, versión de QSB, comando, backend, opciones y resultado. Registre también cualquier transformación desde una hoja de cálculo. En datos externos, añada fuente, licencia, fecha y unidad de análisis. Un documento de solución no siempre contiene el modelo completo; guárdelo por separado.

Los comandos de este manual se ejecutan desde la raíz del repositorio. `swift run qsb` compila si hace falta; `.build/debug/qsb` usa el binario construido. Los ejemplos pequeños permiten comprobar resultados a mano antes de estudiar casos mayores. Cambiar la entrada requiere volver a validar y resolver, no sólo modificar una tabla de presentación.

Observación 2.1. El resultado computacional expresa consecuencias de los datos y supuestos codificados. El analista debe justificar esos supuestos, contrastar la solución con el sistema real y comunicar las limitaciones a quien toma la decisión.

2.5. Comparación y sensibilidad

Distinga verificación (el programa calcula lo especificado) y validación del modelo (lo especificado representa suficientemente el sistema). Compruebe balances y restricciones con los valores devueltos. Después varíe demanda, costes o probabilidades, manteniendo sus distribuciones y unidades coherentes. Un cambio de recomendación ante una perturbación pequeña puede revelar una decisión frágil; un empate puede requerir criterios secundarios fuera del modelo. No extrapole una conclusión más allá del intervalo analizado.

Parte II

Optimización y decisiones

Capítulo 3

Programación lineal

3.1. Qué representa un modelo lineal

La programación lineal (PL) asigna recursos escasos entre actividades cuando el objetivo y las restricciones son lineales [Dantzig, 1963, Bertsimas and Tsitsiklis, 1997]. Una variable puede representar unidades producidas, horas asignadas o cantidades transportadas. Su significado y unidad deben fijarse antes de escribir coeficientes.

Definición 3.1. Un modelo lineal continuo tiene la forma

$$\operatorname{opt}_{x\in\mathbb{R}^n} c^\top x, \qquad a_i^\top x \ \{\le, =, \ge\}\ b_i \quad (i = 1, \dots, m), \qquad \ell_j \le x_j \le u_j.$$

El operador indica minimizar o maximizar. Una cota puede ser inexistente; una variable libre admite valores positivos y negativos.

La linealidad presupone proporcionalidad y aditividad: duplicar una actividad duplica su consumo y su contribución, y las contribuciones se suman sin efectos cruzados. La formulación continua admite divisibilidad. Además, los datos son deterministas durante el horizonte modelado. Descuentos, arranques, lotes indivisibles o rendimientos variables requieren otra formulación. Validar un archivo verifica su estructura y reglas matemáticas; no demuestra que esos supuestos representen bien la organización.

3.2. Ejemplo: mezcla de producción

Una planta fabrica dos productos divisibles, X_1 y X_2. Su contribución unitaria es 50 y 60 unidades monetarias. Cada unidad utiliza dos recursos según la tabla siguiente; las capacidades corresponden al mismo período que la producción. El modelo es

Tabla 3.1: Datos de la mezcla de producción

Recurso	Por unidad de X_1	Por unidad de X_2	Capacidad
Recurso 1	2	3	180
Recurso 2	3	2	150

$$\begin{aligned} \text{máx} \quad & 50X_1 + 60X_2 \\ \text{sujeto a} \quad & 2X_1 + 3X_2 \le 180, \\ & 3X_1 + 2X_2 \le 150, \\ & X_1, X_2 \ge 0. \end{aligned}$$

Las capacidades son restricciones duras: no se autoriza excederlas. No se han incluido mínimos de demanda ni costos fijos; atribuirlos a este resultado alteraría el problema.

3.2.1. Entrada y ejecución reproducibles

El modelo completo está en `manual/examples/lp_sample.json`. Las claves `sense`, `variableNames`, `objectiveCoefficients` y `constraints` definen el objetivo y las filas. Cada fila incluye nombre, coeficientes en el orden de variables, relación y `rhs`. Las cotas son `lowerBounds` y `upperBounds`; `null` en una cota superior significa que no existe un máximo explícito. Los tipos del ejemplo son `continuous`. Ejecute desde la raíz del repositorio:

```
swift run qsb solve-json manual/examples/lp_sample.json --backend native
```

Esta ruta específica devuelve `objectiveValue` y el diccionario `variableValues`. No incluye un informe de sensibilidad.

En macOS, abra el JSON normalizado en la familia `Linear / Integer Programming`. La vista `Definition` presenta `Model`, `Variables and objective` y las restricciones. Para introducirlo manualmente, seleccione `Maximize`, cree dos variables con coeficientes 50 y 60, tipo `Continuous`, cota inferior cero y sin cota superior; después introduzca las dos filas con relación $\leq$. Use `Validate` y corrija los diagnósticos antes de pulsar `Run LP`. Los atajos del editor son Comando–Mayúsculas–V para validar y Comando–R para resolver. `Copy Table` y `Paste Table` permiten trasladar tablas con una hoja de cálculo. Conserve el orden y los encabezados de las columnas; revise el modelo después de pegar. En ventanas estrechas, el editor puede requerir desplazamiento horizontal.

3.2.2. Resultado e interpretación

Tabla 3.2: Óptimo lineal verificado con QSB

Cantidad	Valor
X_1	18
X_2	48
Contribución total	3780

La vista de solución muestra el objetivo, las variables y el detalle de las restricciones. Compruebe numéricamente la factibilidad:

$$2(18) + 3(48) = 180, \qquad 3(18) + 2(48) = 150.$$

Ambos recursos quedan agotados; sus holguras $b_i - a_i^\top x$ son cero. Una restricción activa es una igualdad en el óptimo; una holgura positiva indica capacidad no utilizada. Esta interpretación depende de la relación: para $a_i^\top x \geq b_i$, el exceso es $a_i^\top x - b_i$.

La región factible es un polígono con vértices $(0, 0)$, $(50, 0)$, $(18, 48)$ y $(0, 60)$. Sus objetivos son 0, 2500, 3780 y 3600. Esta comprobación geométrica independiente confirma el resultado para el problema de dos variables. En dimensiones mayores, enumerar vértices a mano no resulta práctico.

3.3. Dualidad y sensibilidad: qué puede deducirse

Para este máximo con restricciones $Ax \leq b$, $x \geq 0$, el dual es

$$\text{mín } 180y_1 + 150y_2, \qquad 2y_1 + 3y_2 \geq 50, \quad 3y_1 + 2y_2 \geq 60, \quad y_1, y_2 \geq 0.$$

Como las dos variables de producción son positivas en el óptimo, las dos restricciones duales se satisfacen como igualdades. De ellas se obtiene $y_1 = 16$, $y_2 = 6$ y $180(16) + 150(6) = 3780$. Los valores duales certifican una cota superior igual al objetivo alcanzado.

Mientras siga siendo óptima esta base, una unidad adicional del recurso 1 vale 16 unidades monetarias y una del recurso 2 vale 6. Son valores marginales locales, no precios válidos para cualquier ampliación. Por ejemplo, al cambiar solo la primera capacidad a $180 + \delta$, la intersección pasa a

$$X_1 = 18 - \tfrac{2}{5}\delta, \qquad X_2 = 48 + \tfrac{3}{5}\delta.$$

Ambas cantidades siguen siendo no negativas para $-80 \leq \delta \leq 45$, intervalo en el que el objetivo es $3780 + 16\delta$. En los extremos cambia la estructura de la base; fuera del intervalo hay que resolver de nuevo. Esta derivación es un cálculo académico del ejemplo: QSB no presenta actualmente precios sombra ni rangos de sensibilidad en la salida LP específica ni en su vista nativa.

3.4. Método, backends y límites

El backend `native` utiliza simplex de dos fases: busca primero una solución factible y después mejora el objetivo. Es determinista y educativo, con aritmética de punto flotante. Un óptimo matemático no implica precisión simbólica. Escalas muy diferentes entre coeficientes pueden deteriorar los cálculos; use unidades coherentes y compruebe residuos.

`validate` ejecuta las comprobaciones sin calcular un óptimo. `external` solicita HiGHS, instalado independientemente y descubierto mediante `QSB_HIGHS_PATH` o `PATH`; si no está disponible, la petición falla explícitamente. No hay sustitución silenciosa por el backend nativo. El motor educativo no ofrece las herramientas de preprocesamiento ni las garantías de escala de un motor industrial. Para intercambiar el modelo con otro optimizador se puede generar MPS:

```
swift run qsb export-mps manual/examples/lp_sample.json > /tmp/qsb-lp.mps
```

El MPS conserva nombres mediante una adaptación determinista y expresa cotas y dominios; el JSON sigue siendo la representación normalizada de QSB.

Distinga tres situaciones: un modelo inválido contiene datos incoherentes; uno infactible no tiene ninguna decisión que satisfaga las restricciones; uno no acotado admite mejoras ilimitadas del objetivo. Agregar una cota arbitraria para hacer desaparecer un error cambia el significado del modelo. Revise las unidades, los signos y los límites que realmente impone el caso.

3.5. Ejercicios

1. Aumente la capacidad del recurso 1 a 190. Antes de ejecutar, prediga $X_1 = 14$, $X_2 = 54$ y objetivo 3940 mediante la sensibilidad anterior; confirme las restricciones con QSB.

2. Reduzca la capacidad del recurso 2 a 120 y resuelva. Explique por qué ya no debe extrapolarse indefinidamente una valoración marginal.

3. Añada $X_1 \geq 60$. Identifique la contradicción con el recurso 2 y compare un modelo estructuralmente válido con un problema infactible.

4. Cambie las variables a enteras y estudie el capítulo siguiente. ¿Por qué en el ejemplo original no cambia el óptimo?

Capítulo 4

Programación entera y mixta

4.1. Decisiones indivisibles

La programación entera exige que algunas decisiones adopten valores discretos: proyectos seleccionados, turnos, lotes o instalaciones abiertas. Un modelo mixto entero lineal conserva objetivo y restricciones lineales, pero combina variables continuas y enteras [Nemhauser and Wolsey, 1988] y [Wolsey, 1998]. La integridad es parte del modelo, no una operación de presentación del resultado. Redondear un óptimo continuo puede incumplir restricciones o perder optimalidad.

Una variable entera representa cantidades indivisibles; una binaria representa una decisión sí/no. En QSB los tipos normalizados son `continuous`, `integer` y `binary`. Todos comparten coeficientes, relaciones y cotas del modelo lineal. En un modelo de selección, una binaria y_j puede activar una cantidad continua x_j mediante $0 \le x_j \le M_j y_j$. La constante M_j debe ser una cota justificable: un número excesivamente grande debilita la relajación y puede perjudicar la estabilidad numérica.

4.2. Ejemplo: cobertura mínima con actividades enteras

Dos actividades generan coberturas de dos requerimientos. Cada unidad de la primera cuesta 2.5 unidades monetarias y cada unidad de la segunda cuesta 2. Sus aportaciones son $(6, 3)$ y $(3, 5)$; se necesitan al menos 200 unidades del primer requerimiento y 180 del segundo. Las actividades solo pueden contratarse por unidades completas:

$$\begin{aligned} \text{mín} \quad & 2{,}5X_1 + 2X_2 \\ \text{sujeto a} \quad & 6X_1 + 3X_2 \ge 200, \\ & 3X_1 + 5X_2 \ge 180, \\ & X_1, X_2 \in \mathbb{Z}_+. \end{aligned}$$

El costo es aditivo y las aportaciones se mantienen constantes: no se han modelado descuentos ni cambios de productividad. El caso completo se encuentra en `manual/examples/ilp_sample.json`; ambas variables tienen `variableTypes` igual a `integer` y cota inferior cero.

4.2.1. Ejecución en CLI y macOS

Desde la raíz del repositorio:

```
swift run qsb solve-json-ilp manual/examples/ilp_sample.json --backend native
```

Use la ruta ILP específica para respetar los dominios enteros; resolver solo la relajación no responde a la decisión indivisible.

En macOS se utiliza el mismo editor `Linear / Integer Programming` del capítulo 3. Abra el JSON o introduzca las dos variables con `Minimize`, coeficientes 2.5 y 2, tipo `Integer`, cota inferior cero y ninguna cota superior explícita. Introduzca las dos restricciones con relación $\geq$, valide y pulse `Run ILP`. El nombre del botón cambia según los tipos de variables. La solución presenta las cantidades y las evaluaciones de las restricciones; conserve el JSON exportado para revisar los dominios originales.

4.2.2. Óptimo y comprobación

QSB devuelve $X_1 = 22$, $X_2 = 23$ y costo 101. La factibilidad se verifica con

$$6(22) + 3(23) = 201 \geq 200, \qquad 3(22) + 5(23) = 181 \geq 180.$$

Los excesos de cobertura son de una unidad en cada requerimiento. El costo corresponde a $2{,}5(22) + 2(23) = 101$, y ambas cantidades son enteras.

Para comprender la diferencia con la relajación, permita temporalmente variables continuas. La intersección de las dos fronteras es

$$X_1 = \frac{460}{21} \approx 21{,}904762, \qquad X_2 = \frac{160}{7} \approx 22{,}857143,$$

con costo $2110/21 \approx 100{,}476190$. En minimización, relajar la integridad solo puede reducir el costo óptimo o dejarlo igual; constituye una cota inferior para el problema entero. La diferencia absoluta es $11/21 \approx 0{,}523810$. El redondeo hacia arriba produce en este caso la solución entera óptima, pero es una coincidencia del caso, no una regla general.

Hay también una comprobación breve del costo entero. Para cualquier solución factible, el costo es al menos $2110/21$, y pertenece a la malla de múltiplos de 0.5. Solo habría que descartar el costo 100.5 para probar que 101 es el mínimo. Si costara 100.5, entonces $5X_1 + 4X_2 = 201$. Sustituyendo en las dos coberturas se obtiene simultáneamente $X_1 \geq 197/9$ y $X_1 \leq 285/13$; el estrecho intervalo $[21{,}888889, 21{,}923077]$ no contiene ningún entero. Por tanto, el costo 100.5 no es alcanzable y la solución de QSB certifica el siguiente valor posible, 101.

4.3. Ramificación y acotación

El algoritmo nativo resuelve una relajación LP en cada nodo. Si la solución es entera, sirve como candidata. Si una variable toma un valor fraccionario v, se crean dos problemas, $x_j \leq \lfloor v \rfloor$ y $x_j \geq \lceil v \rceil$, que cubren sus posibilidades enteras. Una rama infactible se descarta; una cuya cota no mejora la mejor solución conocida también se descarta. Explorar todas las ramas relevantes permite establecer optimalidad dentro de las tolerancias numéricas.

QSB usa una búsqueda determinista y un límite predeterminado de 10000 nodos. Si se excede, informa de un modelo no soportado por ese recorrido; no debe interpretarse como una solución óptima ni como una prueba de infactibilidad. El backend educativo carece de las técnicas de cortes y preprocesamiento de un motor MILP industrial. El tiempo puede crecer rápidamente aunque el archivo sea pequeño; proporcionar cotas físicas válidas puede mejorar la formulación, pero nunca debe excluir decisiones factibles.

`validate` comprueba el modelo sin resolverlo. `external` utiliza el HiGHS descubierto en el equipo para LP/MIP y requiere una instalación independiente. El backend seleccionado no cambia el significado de las variables; si no está disponible, la petición falla. El comando `export-mps` admite este JSON y expresa sus dominios enteros, sus cotas y las filas lineales. Esta capacidad de MILP no implica que HiGHS admita programación cuadrática mixta entera; son clases diferentes.

4.4. Interpretación y errores frecuentes

Compare costos solo entre soluciones factibles del mismo modelo. La cota de la relajación mide el costo idealizado al admitir fracciones, pero no una alternativa implementable. En modelos binarios compruebe que cada valor representa la acción prevista; en modelos mixtos distinga cantidades continuas de decisiones de activación. Una solución fraccionaria obtenida al cambiar los tipos a continuos responde a otro problema.

No multiplique una variable binaria por una continua dentro de una fila LP: es una expresión no lineal. Las reglas de activación requieren restricciones lineales auxiliares. Tampoco convierta una variable continua a entera solo para que la pantalla muestre números redondos: el dominio responde a la unidad de decisión.

4.5. Ejercicios

1. Duplique el JSON del ejemplo y cambie los dos tipos a `continuous`. Resuelva con `solve-json` y contraste la cota inferior calculada arriba con el resultado entero.

2. Imponga $X_1 \leq 21$. Calcule el mínimo X_2 que satisface ambas coberturas, y compruebe el nuevo costo con QSB.

3. Formule una selección de tres proyectos con costos 4, 6 y 7, beneficios 5, 8 y 10, presupuesto 10 y variables binarias. Determine por enumeración qué combinación es óptima y después verifíquela.

4. Explique por qué redondear hacia abajo las dos cantidades de la relajación del ejemplo incumple los requerimientos. ¿Qué cambiaría si las restricciones fueran capacidades máximas?

Capítulo 5

Metas, programación cuadrática y no lineal

Este capítulo reúne tres extensiones distintas de la programación lineal. Las metas ordenan varios objetivos; la programación cuadrática añade curvatura al objetivo manteniendo restricciones lineales; la programación no lineal admite expresiones generales. Elegir una familia cambia tanto la formulación como la garantía que puede ofrecer el solver. No basta con comparar el número que aparece como objetivo: hay que revisar prioridades, factibilidad y exactitud.

5.1. Programación por metas

5.1.1. Prioridades y desviaciones

Una organización puede buscar primero el cumplimiento de una meta y después optimizar otra magnitud, sin sacrificar lo ya conseguido. En la programación lexicográfica se optimiza $g_1(x)$, se fija su valor óptimo y se optimiza $g_2(x)$ dentro de ese conjunto; se continúa así en el orden declarado. No se suman automáticamente objetivos con grandes pesos. El orden expresa una decisión de política: una mejora de una prioridad posterior nunca compensa un empeoramiento de una anterior.

Para una meta $a^\top x = t$, las variables de desviación satisfacen

$$a^\top x + n - p = t, \qquad n, p \geq 0.$$

n mide el déficit y p el exceso. Minimizar $n + p$ penaliza ambas direcciones; minimizar solo n tolera el exceso. Los coeficientes y las unidades deben indicar la penalización deseada. QSB no crea estas ecuaciones por inferencia: el usuario declara variables, objetivos y restricciones.

5.1.2. Ejemplo: proteger un compromiso de producción

Hay capacidad para diez unidades entre los productos A y B. Se desea primero aproximar A a ocho unidades y después maximizar B:

$$\begin{aligned} &\text{Prioridad 1: } \min(n_1 + p_1), \\ &\text{Prioridad 2: } \max B, \\ &A + B \leq 10, \qquad A + n_1 - p_1 = 8, \\ &A, B, n_1, p_1 \geq 0. \end{aligned}$$

La prioridad no obliga por sí misma a satisfacer exactamente la meta si resulta imposible; minimiza su desviación sujeta a las restricciones duras. En este ejemplo sí es posible alcanzar desviación cero. Una vez fijada, $A = 8$, por lo que el máximo posterior es $B = 2$.

El archivo `manual/examples/opt_goal.json` contiene todas las variables, las dos filas y la lista ordenada `goals`; cada prioridad tiene nombre, `sense` y `coefficients`. Las cotas y los tipos se declaran en arreglos alineados con `variableNames`. Ejecute:

```
swift run qsb validate-goal-json manual/examples/opt_goal.json
swift run qsb solve-goal-json manual/examples/opt_goal.json --backend native
```

El documento de salida conserva el modelo, los metadatos `backend` y la solución.

En `goalOutcomes` se muestran prioridad, sentido y valor. La capacidad queda activa y

Tabla 5.1: Resultado lexicográfico del caso de producción

Magnitud	Valor
Prioridad 1: desviación total	0
Prioridad 2: producción de B	2
A	8
n_1, p_1	0, 0

la meta se cumple. Si se invierte el orden, el primer objetivo produce $B = 10$; con ese valor fijado, $A = 0$, $n_1 = 8$ y la desviación es ocho. Este contraste permite discutir quién decide la precedencia y qué compromiso protege.

5.1.3. Flujo en macOS y carácter del solver

Abra el archivo normalizado en `Goal Programming`. En `Definition`, revise variables y cotas, la lista de prioridades y las restricciones lineales. Los controles de subir y bajar cambian el orden de una prioridad. Introduzca la ecuación de desviación como una igualdad y verifique el signo negativo de p_1. Pulse `Validate` y `Run`; la vista nativa de prioridades muestra los objetivos ordenados, las cantidades finales y la información del backend. El JSON ofrece el registro exportable completo. Las tablas admiten copiar y pegar; las columnas de coeficientes deben coincidir con el orden actual de variables. Las tablas grandes usan selección de fila y grupos de coeficientes.

El backend nativo resuelve una secuencia de LP o problemas enteros, añadiendo una igualdad para conservar cada óptimo previo. El caso continuo aparece como `exact`; este término caracteriza el método, sujeto a tolerancias de punto flotante. Las metas enteras usan un recorrido educativo de escala pequeña, identificado como `fixtureScale`, y dependen de las limitaciones de ramificación y acotación. La relajación especial de pares de desviación enteros solo se aplica cuando ecuaciones integrales justifican que los residuos serán enteros; no elimina la integridad de las decisiones.

`validate` no resuelve. `external` ejecuta cada prioridad con HiGHS si está instalado; QSB conserva el orden y el fijado lexicográfico. Si falta el ejecutable, hay un error explícito. Una prioridad posterior infactible, un objetivo no acotado o el agotamiento de límites requieren revisar el diagnóstico, no interpretar resultados parciales como el plan final.

5.2. Programación cuadrática

5.2.1. Convención del objetivo y supuestos

QSB representa el objetivo como

$$f(x) = x^\top Q x + c^\top x,$$

con restricciones lineales, cotas y tipos de variables. El JSON utiliza `quadraticMatrix` para Q y `linearCoefficients` para c. La convención no contiene el factor 1/2. Para una matriz simétrica, el coeficiente del producto $x_i x_j$, $i \neq j$, es $2Q_{ij}$: un término $4XY$ se representa con $Q_{12} = Q_{21} = 2$. No coloque 4 en ambas celdas, porque duplicaría el efecto.

Una matriz simétrica definida positiva produce un objetivo estrictamente convexo; su mínimo sobre una región convexa, si existe, es único. Una matriz definida negativa produce el caso equivalente de máximo estrictamente cóncavo. Las condiciones de estacionariedad y factibilidad permiten entonces identificar el óptimo global. Una matriz indefinida cambia el problema y no queda cubierta por esa garantía.

5.2.2. Ejemplo: ajuste con capacidad limitada

Se desea aproximar dos cantidades a 2 y 3, respectivamente, con capacidad total cuatro. Eliminando la constante 13 de la distancia cuadrática se obtiene

$$\begin{aligned} \text{mín} \quad & X^2 + Y^2 - 4X - 6Y \\ \text{sujeto a} \quad & X + Y \leq 4, \qquad X, Y \geq 0. \end{aligned}$$

La matriz es la identidad y el vector lineal es $(-4, -6)$. La eliminación de la constante conserva las cantidades óptimas, pero cambia el valor objetivo reportado: la distancia original es $f(X, Y) + 13$.

El modelo está en `manual/examples/opt_qp.json`. Ejecute:

```
swift run qsb validate-qp-json manual/examples/opt_qp.json
swift run qsb solve-qp-json manual/examples/opt_qp.json --backend native
```

El óptimo es $X = 1{,}5$, $Y = 2{,}5$, $f = -12{,}5$, con `Capacity` en `activeConstraints`. La distancia cuadrática original es 0.5. Un objetivo negativo es perfectamente válido: procede de la constante omitida y no representa una distancia negativa.

La solución sin capacidad sería $(2, 3)$, que no es factible. Con multiplicador $\lambda \geq 0$, las ecuaciones de estacionariedad son

$$2X - 4 + \lambda = 0, \qquad 2Y - 6 + \lambda = 0.$$

La capacidad activa exige $X + Y = 4$, de donde $\lambda = 1$, $X = 1{,}5$ y $Y = 2{,}5$. Las cotas inferiores no son activas. La convexidad estricta y las condiciones anteriores certifican el mínimo global del modelo continuo. QSB muestra las restricciones activas, pero esta derivación del multiplicador es una comprobación académica independiente.

5.2.3. Flujo en macOS y límites

Abra el JSON en `Quadratic Programming`. `Definition` separa variables y cotas, matriz cuadrática y restricciones lineales. Introduzca los coeficientes lineales en las filas de variables y la identidad en la matriz; confirme los nombres de sus dos ejes. Añada `Capacity` con coeficientes $(1, 1)$, relación $\leq$ y término derecho 4. Valide y pulse `Run`. La solución nativa presenta objetivo, variables, cotas, restricciones activas y exactitud; el JSON permite revisar el documento.

`Copy Table` y `Paste Table` trasladan la matriz completa. Reordenar variables conocidas conserva ambos ejes por nombre; los encabezados pegados deben coincidir con el orden vigente. Para más de doce variables, el editor muestra una fila y grupos de columnas, aunque el intercambio de la tabla sigue siendo completo. Esta capacidad del editor no implica que el solver nativo admita problemas grandes.

El nativo continuo enumera conjuntos activos y resuelve sistemas KKT para mínimos estrictamente convexos o máximos estrictamente cóncavos. La etiqueta es `exact`, con cálculos

numéricos. El recorrido rechaza más de veinte candidatos de desigualdad, contando las cotas que convierte a desigualdades; el costo de enumerar conjuntos crece rápidamente. El nativo entero admite solo maximización cóncava diagonal con variables no negativas, hasta cuatro variables, y enumera puntos en cotas finitas derivadas. Se identifica como `fixtureScale`; no es un solver MIQP general.

El backend externo HiGHS admite la traducción de los casos continuos estrictamente convexos/cóncavos soportados. QSB adapta la convención mediante una Hessiana $H = 2Q$, porque el solver externo utiliza $\frac{1}{2}x^\top Hx$. La adaptación también conserva sentido, cotas y restricciones, y verifica objetivo y factibilidad al leer la solución. HiGHS no resuelve MIQP en esta integración; las variables enteras cuadráticas requieren el alcance nativo descrito o un motor futuro. El modo `validate` no calcula un óptimo.

5.3. Programación no lineal

5.3.1. Expresiones y dominio

La programación no lineal (PNL) admite un objetivo $f(x)$ y restricciones $g_i(x)\ \{\leq, =, \geq\}\ b_i$. QSB recibe expresiones textuales, variables ordenadas y cotas anulables. El evaluador reconoce suma, resta, producto, división, potencias, paréntesis, multiplicación implícita y `exp`, `log`, `sqrt`, `sin`, `cos`. Use producto explícito en documentos didácticos para facilitar la lectura. Los nombres se reconocen sin distinguir mayúsculas de minúsculas; conviene mantener una escritura uniforme.

La sintaxis válida no garantiza un dominio válido para todos los puntos: `log(X)` requiere $X > 0$, `sqrt(X)` requiere $X \geq 0$ y una división requiere denominador no nulo. Las cotas deben corresponder al caso y proteger el dominio cuando sea posible. Los ángulos de seno y coseno se expresan en radianes. El JSON guarda `objectiveExpression` y, en cada restricción, `expression`, `relation` y `rhs`.

5.3.2. Ejemplo: distancia con restricción

El archivo `manual/examples/opt_nlp.json` plantea

$$\text{mín } (X-3)^2 + (Y-2)^2, \qquad X + Y \leq 4, \qquad 0 \leq X, Y \leq 4.$$

El objetivo es la distancia cuadrática al punto $(3, 2)$, que viola la capacidad. La proyección sobre $X+Y=4$ es (2,5, 1,5) y el valor matemático es 0.5. Aunque este caso podría formularse como QP, sirve para comparar una entrada por expresión y un método aproximado con una solución analítica conocida.

```
swift run qsb validate-nlp-json manual/examples/opt_nlp.json
swift run qsb solve-nlp-json manual/examples/opt_nlp.json --backend native
```

La ejecución verificada devuelve aproximadamente $X = 2{,}500000002$, $Y = 1{,}499999999$, objetivo 0.4999999995 y violación máxima $5{,}0 \times 10^{-10}$. La evaluación de `Capacity` es aproximadamente 4.0000000005. El campo `iterations` registra 361 iteraciones acumuladas de la búsqueda seleccionada; no cuenta todas las evaluaciones de la búsqueda desde los otros puntos iniciales. Este valor describe esta ejecución, no un límite universal ni una medida comparable con iteraciones simplex.

La pequeña violación no autoriza exceder físicamente una capacidad. Es un residuo numérico de la búsqueda; para implementación real, compruebe los valores con la tolerancia y las unidades del caso. La salida contiene `constraintEvaluations`, `maximumViolation`, `iterations`, objetivo y valores de variables, además del modelo y metadatos. Reportar el objetivo sin el residuo ocultaría información relevante.

5.3.3. Uso en macOS y garantía aproximada

Abra el JSON en `Nonlinear Programming`. La definición permite editar el sentido, la expresión objetivo, variables con cotas y restricciones con expresión, relación y término derecho. Introduzca `(X-3)^2+(Y-2)^2`, límites 0 y 4, y `X+Y` con ≤ 4. Use `Validate` antes de `Run`. Una expresión inválida se conserva para corregirla. Variables y restricciones admiten intercambio con hojas de cálculo; cada pegado sustituye una tabla completa cuando es válido. La solución presenta el candidato, límites, evaluaciones y violación máxima, junto con el carácter aproximado y el JSON exportable.

El backend nativo usa diferenciación automática, búsqueda desde varios puntos y penalizaciones progresivas de restricciones. Es determinista, se limita a cuatro variables y se identifica como `approximate`. También contiene una búsqueda de baja dimensión sobre variedades de igualdad para casos pequeños. Estos mecanismos no prueban optimalidad global para funciones no convexas. Un candidato factible puede ser local, y una búsqueda fallida no constituye por sí sola una prueba matemática de infactibilidad.

Las desigualdades estrictas procedentes de importaciones se normalizan a fronteras no estrictas y generan una advertencia estructurada. El campo `normalizedStrictInequalities` registra esa transformación; debe revisarse si la frontera tiene un significado físico excluido por el caso. `validate` diagnostica sin resolver. `external` está explícitamente no disponible para PNL; seleccione el nativo para los problemas educativos soportados.

5.4. Cómo comparar y revisar resultados

Conserve cada entrada junto con su salida y el backend utilizado. En metas, revise el orden y cada valor fijado. En QP, compruebe la convención de la matriz y la curvatura admitida. En PNL, revise dominios, cotas, evaluaciones y violación máxima antes de hablar de una solución aplicable. La validación del archivo y la convergencia de una búsqueda son evidencias distintas.

Las tres familias tienen definición nativa y vista JSON; los campos pendientes siguen editables cuando la validación falla. En las ejecuciones nativas de macOS, la resolución se realiza en segundo plano y se impiden ejecuciones duplicadas mientras hay una en curso. Cambiar el modelo puede dejar obsoleto el resultado de ese cálculo. No existe actualmente un control de cancelación para esos trabajadores. La presencia de etiquetas accesibles en los controles y resultados no sustituye una auditoría completa de VoiceOver y navegación por teclado, que sigue pendiente para estos editores.

5.5. Ejercicios

1. Invierta las prioridades del ejemplo de metas. Compruebe $B = 10$, $A = 0$ y desviación total 8. Explique qué compromiso se ha cambiado.

2. Reduzca la capacidad del ejemplo de metas a 6. Calcule la desviación mínima de la primera prioridad y justifique por qué la segunda no puede recuperar producción de B sin empeorar la primera.

3. En el QP aumente la capacidad a 5. Prediga $(X, Y) = (2, 3)$, objetivo -13 y distancia original cero. Contraste con QSB.

4. Introduzca un término $4XY$ en un objetivo de prueba. Escriba la matriz simétrica correcta y examine su curvatura antes de elegir solver.

5. Reformule el objetivo PNL del ejemplo en forma QP, conservando restricciones y cotas. Compare cantidades y residuos; explique la diferencia entre su objetivo con constante y el objetivo cuadrático sin ella.

6. Proponga una función de una variable con dos mínimos locales. Explique qué evidencia adicional necesitaría para atribuir optimalidad global a un candidato obtenido por el nativo PNL.

Capítulo 6

Modelos de redes

6.1. De una situación a un grafo

Una red representa lugares, estados o personas mediante nodos; los arcos representan conexiones posibles. El significado del número de un arco depende del modelo: costo o distancia en una ruta, capacidad en flujo máximo. Confundir ambas magnitudes cambia el problema [Ahuja et al., 1993]. Los ejemplos de este capítulo son originales y sus archivos están en `manual/examples/`. Todos usan el sobre JSON con `kind` y `model`; no se necesita disponer de archivos de WinQSB. Los comandos se ejecutan desde la raíz del repositorio.

6.2. Ruta más corta y árbol de expansión mínima

Para SPP se busca una ruta dirigida del origen al destino de costo mínimo. Dijkstra exige costos no negativos. En QSB el primer nodo es el origen y el último es el destino. Un arco ausente no equivale a un arco de costo cero. El archivo `plan_spp.json` tiene nodos A, B, C, D y arcos:

Arco	A–B	A–C	B–C	B–D	C–D
Costo	2	5	1	6	2

```
swift run qsb validate-network-json manual/examples/plan_spp.json
swift run qsb solve-network-json manual/examples/plan_spp.json --backend native
swift run qsb solve-network-json manual/examples/plan_mst.json --backend native
```

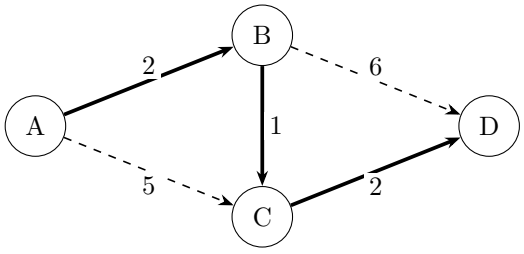


Figura 6.1: Red de ruta más corta: las etiquetas son costos. Las flechas gruesas trazan la ruta óptima; las discontinuas muestran alternativas.

SPP devuelve A → B → C → D y costo $2 + 1 + 2 = 5$. MST conecta todos los nodos sin ciclos, con el menor costo total; no busca una ruta entre extremos. El ejemplo usa las mismas conexiones como aristas no dirigidas y Kruskal elige B–C, A–B, C–D, con costo 5. QSB normaliza conexiones asimétricas de MST al menor costo por pareja; una red desconectada no admite árbol que cubra todos los nodos.

6.3. Flujo máximo

Con capacidades u_{ij}, el flujo debe satisfacer $0 \le x_{ij} \le u_{ij}$ y conservación en nodos intermedios. Se maximiza el flujo neto que sale del origen. No se minimiza costo. En `plan_maxflow.json`, S–A tiene capacidad 3, S–B 2, A–B 1, A–T 2 y B–T 3. Aunque el campo JSON se llama `cost`, en MFP representa capacidad.

```
swift run qsb solve-network-json manual/examples/plan_maxflow.json --backend native
```

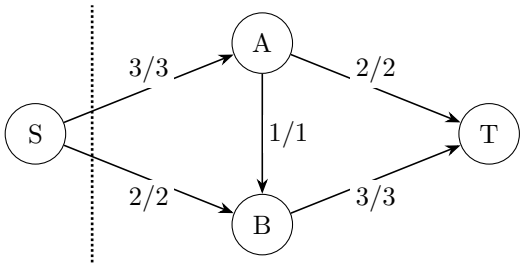


Figura 6.2: Flujo máximo: cada etiqueta indica flujo/capacidad. La línea punteada separa el origen del resto y marca un corte saturado.

Edmonds–Karp obtiene flujo 5: salen 3 y 2 unidades de S y llegan 2 y 3 a T. El corte que separa S del resto tiene capacidad 5 y prueba optimalidad. El primer y último nodo vuelven a determinar los extremos.

6.4. Vendedor viajero

TSP busca un ciclo dirigido que visite cada nodo exactamente una vez y vuelva al inicial. No basta con conectar los nodos: hay que respetar una única gira. El ejemplo `plan_tsp.json` contiene tres nodos, con $c_{AB} = 2$, $c_{BC} = 3$, $c_{CA} = 4$ y costos inversos 4, 5 y 6.

```
swift run qsb solve-network-json manual/examples/plan_tsp.json --backend native
```

La gira A $\to$ B $\to$ C $\to$ A cuesta 9; la inversa cuesta 15. Held–Karp es exacto para el modelo aceptado, pero exponencial; el solver rechaza más de 20 nodos. Un modelo sin ciclo hamiltoniano factible no se repara añadiendo conexiones. La etiqueta `fixtureScale` expresa esta limitación de escala, no una garantía de rapidez cerca del máximo.

6.5. Asignación y transporte

Asignación minimiza $\sum_{ij} c_{ij} x_{ij}$, con cada trabajador asignado a una tarea y cada tarea utilizada a lo sumo una vez. El método húngaro acepta matrices rectangulares con al menos tantas tareas como trabajadores. En `plan_assignment.json`, las filas Ana y Luis tienen costos $(2, 7, 4)$ y $(6, 1, 3)$ para X, Y y Z.

```
swift run qsb solve-network-json manual/examples/plan_assignment.json --backend
    native
swift run qsb solve-network-json manual/examples/plan_transport.json --backend
    native
```

Ana–X y Luis–Y cuestan 3; Z queda libre. No se modelan equipos, varias tareas por persona ni restricciones adicionales.

Transporte permite cantidades continuas y exige balance:

$$\text{mín} \sum_{ij} c_{ij} x_{ij}, \qquad \sum_{j} x_{ij} = s_i, \quad \sum_{i} x_{ij} = d_j, \quad x_{ij} \geq 0.$$

El ejemplo tiene oferta $(4, 6)$, demanda $(5, 5)$ y matriz de costos $\left(\begin{smallmatrix} 1 & 4 \\ 3 & 2 \end{smallmatrix}\right)$. QSB envía 4 de O1 a D1, 1 de O2 a D1 y 5 de O2 a D2; el costo es $4 + 3 + 10 = 17$. Compruebe por separado las sumas de filas y columnas. Un desequilibrio es un error de validación en TP. No se crea automáticamente un origen o destino ficticio en esta familia.

6.6. Flujo de costo mínimo

CNF combina transbordo y costos: en cada nodo, $\sum_j x_{ij} - \sum_j x_{ji} = s_i - d_i$, y minimiza $\sum_{ij} c_{ij} x_{ij}$ con cantidades continuas no negativas. El modelo actual no tiene capacidades ni cotas inferiores por arco. El validador exige costos finitos no negativos en CNF y en las variantes de grafo; asignación admite costos negativos finitos. No modele una subvención como costo negativo en un arco CNF, pues será rechazada. `plan_netflow.json` ofrece 4 en O, exige 4 en D y permite O–M a costo 1, M–D a costo 2 y O–D a costo 5.

```
swift run qsb solve-network-json manual/examples/plan_netflow.json --backend native
```

Las 4 unidades pasan por M y cuestan 12; el envío directo costaría 20. CNF conserva oferta y demanda originales y, cuando no coinciden, emite una advertencia y añade un ajuste ficticio de costo cero al resolver. Revise siempre `balanceAdjustments`: una solución con ajuste no prueba que la demanda real pueda servirse con la oferta original.

6.7. Uso en macOS y elección del backend

Abra el ejemplo mediante la importación de JSON normalizado. En Definition, SPP, MST, MFP y TSP disponen de editor de nodos y arcos; AP y TP de matrices de costos, con oferta y demanda en TP; CNF de nodos, arcos y balances. Los cambios se validan antes de Run. El orden de extremos de SPP/MFP se traduce al orden del JSON; las posiciones visuales no se exportan. La solución Diagram resalta la ruta, árbol, gira o conexiones utilizadas. `All connections` muestra también las conexiones restantes. En redes bipartitas, los grupos corresponden a trabajadores/tareas u orígenes/destinos. Use el panel de detalle para leer valores exactos y JSON para auditar el resultado; el grosor de una conexión no sustituye la cantidad numérica.

Native ofrece algoritmos deterministas para las siete variantes. Validate comprueba el modelo sin resolver. External está disponible para TP y CNF mediante HiGHS; no existe un backend externo de todas las redes. Las soluciones conservan modelo, algoritmo, supuestos y exactitud. El tamaño factible de los LP nativos depende de la formulación; para redes de transporte grandes conviene evaluar HiGHS y verificar su disponibilidad.

6.8. Ejercicios

1. Cambie A–C de 5 a 1 en SPP. Verifique la nueva ruta y explique por qué el árbol mínimo puede responder de otra manera.

2. Reduzca S–A a 1 en MFP. Calcule un corte antes de resolver.

3. Cambie la demanda de D2 a 6 en TP. Observe la validación; después represente deliberadamente una unidad adicional de oferta.

4. En CNF aumente la demanda a 5. Interprete el ajuste ficticio y explique qué dato de negocio falta para aceptar el plan.

Capítulo 7

Decisiones, juegos y programación dinámica

7.1. Elegir bajo incertidumbre

Una decisión combina alternativas controlables con estados del entorno que el analista no controla. Una tabla de pagos organiza el beneficio a_{ij} de la alternativa i en el estado j. Cuando se conocen probabilidades previas p_j, el criterio de valor esperado selecciona

$$i^* \in \arg\max_i \sum_j p_j a_{ij}.$$

Este criterio supone pagos comparables y neutralidad frente al riesgo. Una solución que maximiza el beneficio medio puede incluir pérdidas importantes en un estado concreto. QSB no transforma automáticamente dinero en utilidad ni incorpora la aversión al riesgo del decisor.

7.1.1. Tabla de pagos e información

El archivo `dec_payoff.json` plantea un lanzamiento con demanda Alta (probabilidad 0,4) o Baja (0,6). Los pagos son:

Decisión	Alta	Baja
Lanzar	100	−40
Esperar	20	10

Sin estudiar el mercado, Lanzar obtiene $0{,}4(100)+0{,}6(-40) = 16$, frente a 14 de Esperar. Con información perfecta se elegiría Lanzar en Alta y Esperar en Baja: $0{,}4(100) + 0{,}6(10) = 46$. El valor esperado de información perfecta (EVPI) es $46 - 16 = 30$, un techo para el precio de información bajo este modelo.

El estudio produce un indicador Positivo o Negativo. La matriz de `indicatorLikelihoods` usa filas de indicadores y columnas de estados: $P(+ \mid Alta) = 0{,}8$ y $P(+ \mid Baja) = 0{,}2$. Cada columna representa una distribución sobre los posibles indicadores. No son probabilidades posteriores. La política con estudio es Lanzar si sale Positivo y Esperar si sale Negativo. Su valor esperado es 33,6; por tanto, EVSI (valor de información muestral) es $33{,}6 - 16 = 17{,}6$. Este importe no descuenta el coste del estudio. Si el estudio costase 20, no convendría comprarlo con el criterio de valor esperado.

```
swift run qsb validate-decision-json manual/examples/dec_payoff.json
swift run qsb solve-decision-json manual/examples/dec_payoff.json --backend native
```

La salida contiene `priorExpectedValues`, `indicatorAnalyses`, `bestPriorDecision`, EVSI y EVPI. Las claves conservan sus nombres en inglés. El solver exige probabilidad mayor que 10^{-12} para cada indicador; si no se cumple, falla la resolución en vez de inventar una posterior.

7.1.2. Actualización bayesiana

La regla de Bayes convierte probabilidad previa y verosimilitud en probabilidad posterior:

$$P(s_j \mid e_k) = \frac{P(e_k \mid s_j)p_j}{\sum_\ell P(e_k \mid s_\ell)p_\ell}.$$

El archivo `dec_bayes.json` separa esta operación de cualquier tabla de pagos. Usa los mismos estados y el mismo estudio. QSB obtiene $P(+) = 0{,}44$, $P(-) = 0{,}56$, y las posteriores:

Evidencia	Alta	Baja
Positivo	0,727273	0,272727
Negativo	0,142857	0,857143

```
swift run qsb validate-decision-json manual/examples/dec_bayes.json
swift run qsb solve-decision-json manual/examples/dec_bayes.json
```

`likelihoods` vuelve a usar filas de resultados y columnas de estados. Cambiar el orden de los nombres exige cambiar también las filas o columnas correspondientes. Las probabilidades deben ser finitas, no negativas y consistentes con las distribuciones representadas. Bayes actualiza creencias; por sí solo no indica qué acción tomar.

7.1.3. Árboles y análisis hacia atrás

Un árbol hace explícito el orden temporal de decisiones y observaciones. `rootID` identifica la raíz, cada nodo tiene un `id` único y sus `childIDs` indican sucesores. Los tipos son `decision`, `chance` y `terminal`. Las probabilidades de las ramas de azar se guardan en los nodos hijos, no en una lista del padre.

El análisis hacia atrás asigna al terminal su pago. Un nodo de decisión suma su pago local al máximo valor de sus hijos; uno de azar suma su pago local al valor esperado de los hijos. Los pagos locales se acumulan a lo largo de la trayectoria: no repita en las hojas un coste que ya introdujo en un antecesor.

En `dec_tree.json`, Invertir cuesta 5, da 40 con probabilidad 0,6 y -10 con probabilidad 0,4. Su valor es $-5 + 0{,}6(40) + 0{,}4(-10) = 15$; Conservar da 12. El valor de la raíz es 15 y la política selecciona Invertir.

```
swift run qsb validate-decision-json manual/examples/dec_tree.json
swift run qsb solve-decision-json manual/examples/dec_tree.json
```

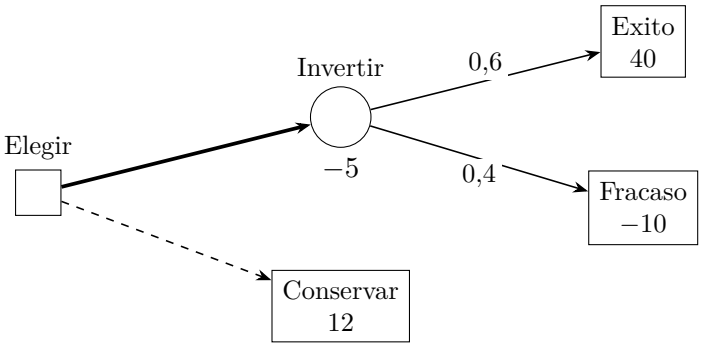


Figura 7.1: Árbol de decisión: el cuadrado permite elegir y el círculo representa azar. La rama gruesa es la política elegida. Las hojas muestran pagos; el costo local de invertir se añade una sola vez. Los nombres conservan los identificadores JSON.

Revise `nodeValues` para cada nodo y `policy` para las decisiones. El método maximiza pagos: para minimizar costes puede usar sus negativos, explicando la convención. QSB advierte y normaliza probabilidades de hijos de azar cuya suma positiva difiere de uno, por compatibilidad con entradas redondeadas. Esa reparación no sustituye una revisión de los datos. Evite ciclos, identificadores inexistentes y probabilidades incoherentes.

7.2. Juegos de suma cero

En un juego de suma cero, el pago del jugador de filas es la pérdida del jugador de columnas. El primero maximiza y el segundo minimiza. Las estrategias mixtas son distribuciones de probabilidad sobre acciones; no representan cantidades fraccionarias de una acción en una partida.

Para una matriz A, QSB resuelve problemas lineales equivalentes. El jugador de filas busca $\max v$ sujeto a $A^T p \geq v\mathbf{1}$, $\sum_i p_i = 1$, $p_i \geq 0$. El jugador de columnas busca el problema dual de minimización. Esto presupone elección simultánea y pagos conocidos. `dec_game.json` usa

$$A = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}.$$

Ambos jugadores asignan probabilidad 0,5 a A y B, y el valor del juego es cero. La salida numérica puede mostrar $4{,}44 \times 10^{-16}$ por redondeo.

```
swift run qsb validate-decision-json manual/examples/dec_game.json
swift run qsb solve-decision-json manual/examples/dec_game.json
```

Lea `rowStrategy`, `columnStrategy` y `value`. El método nativo usa el solver lineal compartido. No interpreta esta solución como un equilibrio de un juego general de suma no nula ni de un juego secuencial.

7.3. Programación dinámica: estado, etapa y acción

La programación dinámica divide un problema en etapas. El estado resume la información necesaria para valorar las decisiones futuras; la acción cambia ese estado y produce un coste o retorno local. El principio de optimalidad permite construir una política óptima a partir de subproblemas. Un estado que omite información relevante rompe ese principio.

QSB implementa tres modelos discretos y deterministas: mochila acotada, ruta acíclica por etapas y producción con inventario. Las entradas usan `kind` y `model`. Las salidas incluyen un resultado tipado, `assumptions` y `trace`. Cada fila de la traza describe etapa, estado, acción,

siguiente estado y valor *local*; ese valor no es necesariamente la función de valor óptimo del subproblema.

7.3.1. Mochila acotada

Cada artículo i consume w_i unidades de capacidad y aporta r_i por unidad; hay como máximo b_i unidades disponibles. La formulación es

$$\max \sum_i r_i x_i, \qquad \sum_i w_i x_i \le C, \qquad x_i \in \{0, \dots, b_i\}.$$

Una recurrencia con capacidad restante c es $F_i(c) = \max_q\{qr_i + F_{i+1}(c - qw_i)\}$, para cantidades disponibles que caben. La capacidad y los consumos son enteros; los retornos pueden ser reales finitos. No convierta medidas físicas continuas a enteros sin justificar la unidad de discretización.

En `dec_knapsack.json`, $C = 7$ y los artículos (b_i, w_i, r_i) son A $(2, 3, 5)$, B $(3, 2, 3)$ y C $(1, 4, 7)$. QSB selecciona una A y una C: capacidad 7 y retorno 12. La alternativa una A y dos B ocupa también 7, pero retorna 11. La selección codifica cantidades; la traza incluye también la decisión de tomar cero unidades de B.

```
swift run qsb validate-dp-json manual/examples/dec_knapsack.json
swift run qsb solve-dp-json manual/examples/dec_knapsack.json
```

En macOS, Bounded Knapsack permite editar capacidad, disponibilidades, consumos y retornos, así como copiar o pegar la tabla. Summary ofrece métricas, barras y una tabla de selección; la traza permite explicar la política.

7.3.2. Ruta por etapas (Stagecoach)

La lista `nodes` declara un orden topológico: el primer nodo es origen y el último destino. Todo arco debe avanzar en ese orden y tener coste finito no negativo. El método aplica

$$V(i) = \min_{(i,j)}\{c_{ij} + V(j)\}, \qquad V(\text{destino}) = 0.$$

`dec_route.json` contiene Origen–Norte (4), Origen–Sur (2), Norte–Destino (3) y Sur–Destino (6). El resultado es Origen–Norte–Destino, con coste 7; elegir el arco inicial más barato daría 8.

```
swift run qsb validate-dp-json manual/examples/dec_route.json
swift run qsb solve-dp-json manual/examples/dec_route.json
```

Un grafo estructuralmente válido puede carecer de ruta hasta el destino; eso se detecta al resolver. Este modelo no acepta ciclos ni arcos hacia atrás: para otros grafos consulte el capítulo de redes.

7.3.3. Producción e inventario

El estado al inicio de un periodo es el inventario I_{t-1}; la acción es producción entera P_t. Se satisface demanda determinista d_t con $I_t = I_{t-1} + P_t - d_t \ge 0$. Las capacidades limitan P_t e I_t. QSB supone inventario inicial y final cero, sin pedidos pendientes. Minimiza

$$\sum_t \left[K_t \mathbf{1}_{P_t>0} + c_t P_t + h_t I_t\right].$$

El coste de posesión se aplica al inventario final del periodo, no al inicial. La recurrencia evalúa producciones factibles y añade el coste óptimo futuro.

`dec_production.json` tiene demandas 2 y 3, capacidad de producción 5, capacidad de almacén 3, preparación 10, coste unitario 2 y posesión 1 en ambos periodos. QSB produce 5 al inicio, conserva 3 y no produce en el segundo periodo. El coste es $10 + 2(5) + 1(3) = 23$, frente a 30 si prepara y produce en cada periodo. Las decisiones detallan inventario inicial, producción, demanda, inventario final y coste local.

```
swift run qsb validate-dp-json manual/examples/dec_production.json
swift run qsb solve-dp-json manual/examples/dec_production.json
```

La validación de campos no prueba que exista un plan factible. Capacidades insuficientes pueden causar fallo de resolución. Este modelo no representa varios productos, inventario inicial configurable, faltantes o demanda aleatoria.

7.4. Uso, metadatos y límites comunes

Las cuatro variantes de decisiones tienen editores nativos en macOS. El árbol permite editar nodos y enlaces; eliminar hojas o añadir terminales conserva una estructura más fácil de revisar. Stagecoach y producción/inventario pueden editarse mediante el formulario estructurado o JSON; el resultado tiene vista nativa de ruta o series por periodo. Use Validation antes de Run y consulte el JSON cuando necesite conservar todos los campos.

El backend nativo de decisiones declara `exact`: calcula el criterio especificado con aritmética numérica, dentro del modelo y la escala educativa. Las tres variantes dinámicas declaran `fixtureScale`; sus políticas son las obtenidas por las recurrencias implementadas, con discretización y consumo de memoria que crecen con capacidad, inventario y número de etapas. No existe una promesa de rendimiento industrial. `-backend validate` devuelve validación y no una solución; el backend externo no está disponible para estas familias. Guarde el modelo junto a la salida de programación dinámica: su documento de solución conserva título y supuestos, pero no el modelo completo.

7.5. Ejercicios y comprobaciones

1. Cambie $P(Alta)$ de 0,4 a 0,3, ajustando también Baja. Calcule primero la mejor decisión sin información y vuelva a resolver. ¿Por qué puede cambiar EVSI aunque no cambie la calidad del estudio?

2. Calcule el precio máximo del estudio del ejemplo y compárelo con EVPI. Explique qué supuestos impedirían usar ese precio en una decisión real.

3. En el árbol, eleve el coste de Invertir a 9. El valor de esta rama será 11 y la decisión de raíz pasará a Conservar, con valor 12.

4. En la mochila, cambie la capacidad a 6. Verifique el retorno 10 con dos unidades A y explique por qué el cociente retorno/consumo no basta para asegurar optimalidad.

5. En la ruta, haga que Sur–Destino cueste 4. La ruta por Sur costará 6. Añada después un arco hacia atrás y examine el diagnóstico de validación.

6. Reduzca el coste de preparación de producción a 1 en ambos periodos. Producir 2 y 3 por separado cuesta 12, frente a 14 al acumular inventario. Compare cada balance de inventario y la traza.

Parte III

Planificación de operaciones

Capítulo 8

Modelos de inventarios

8.1. Qué decisión se modela

Una política de inventario combina cuánto reponer y cuándo hacerlo. El costo mínimo depende de las unidades de tiempo, del patrón de demanda y del costo de faltantes; no basta con introducir una demanda media [Harris, 1913, Silver et al., 1998]. QSB distingue EOQ/EPQ, descuentos por cantidad, vendedor de periódicos, lotes por períodos y cuatro políticas de revisión estocástica. Todos utilizan el sobre JSON `kind`/`model` y el backend nativo o de validación. No hay backend externo de inventarios.

8.2. EOQ y producción gradual

Con demanda constante D, costo fijo S por pedido y mantenimiento H por unidad y período, sin faltantes, el costo relevante es

$$C(Q) = DS/Q + HQ/2, \qquad Q^* = \sqrt{2DS/H}.$$

La adquisición Dc se suma al costo total, pero no altera el óptimo si el precio unitario c es constante. El plazo ℓ produce el punto de reorden $r = D\ell$. Tasas, plazo y mantenimiento deben usar el mismo período. Para producción gradual a tasa $p > D$, el inventario medio es $Q(1 - D/p)/2$ y

$$Q^*_{EPQ} = \sqrt{\frac{2DS}{H(1 - D/p)}}.$$

El cálculo es continuo: comprar cajas o unidades enteras requiere comparar cantidades factibles. QSB no introduce automáticamente restricciones de embalaje.

8.2.1. Ejemplo: reposición de cuadernos

El archivo `manual/examples/prob_eoq.json` contiene $D = 1200$ unidades/año, $S = 30$, $H = 2$, $c = 5$, plazo 0,01 años y cantidad actual de pedido 100. Desde la raíz del repositorio:

```
.build/debug/qsb validate-inventory-json manual/examples/prob_eoq.json
.build/debug/qsb solve-inventory-json manual/examples/prob_eoq.json
```

Se obtiene $Q^* = 189{,}736660$, 6,324555 pedidos/año, ciclo 0,158114 años y punto de reorden 12. Ordenar y mantener cuestan 189,736660 cada uno; el costo relevante es 379,473319 y el total 6379,473319. Pedir 100 cuesta 6460: el ahorro anual del modelo es 80,526681, no el valor completo de las compras. El documento de salida incluye la comparación en `knownQuantity` y exactitud `closedForm`. En `prob_epq.json`, $p = 2400$ eleva el lote óptimo a 268,328157, con costo relevante 268,328157.

8.3. Descuentos sobre todas las unidades

QSB evalúa cada tramo de precio y sus cantidades factibles, incluyendo adquisición, preparación y mantenimiento. El descuento se aplica a todas las unidades del pedido; no es una tarifa marginal por bloques. El mantenimiento es un costo por unidad, no un porcentaje recalculado a partir del precio. `prob_descuento.json` añade un descuento del 10 % a partir de 300 unidades. Con `solve-inventory-json`, resulta elegido $Q = 300$, precio 4,5, costo de preparación 120, mantenimiento 300, adquisición 5400 y total 5820. El costo relevante aumenta a 420, pero el ahorro en adquisición hace preferible el tramo. La tabla de candidatos y `optimum` explican la decisión; la enumeración de tramos es exacta dentro del modelo continuo.

8.4. Vendedor de periódicos

Se decide una sola reposición para un horizonte sin recuperación posterior. Para precio v, adquisición c, salvamento s y penalización adicional por faltante b, los costos marginales son $C_u = v - c + b$ y $C_o = c - s$. La fracción crítica es

$$F(q^*) = \frac{C_u}{C_u + C_o}.$$

QSB admite demanda normal, calcula su cuantil y los valores esperados de ventas, sobrantes, faltantes y beneficio. La probabilidad de cubrir toda la demanda es el nivel de servicio del ciclo; no es la fracción esperada de demanda servida. En `prob_newsboy.json`, media 100, desviación 20, $v = 5, c = 2, s = 1, b = 1$ y costo fijo cero dan fracción crítica 0,8, pedido 116,832425, ventas esperadas 97,767247, sobrantes 19,065178, faltantes 2,232753 y beneficio 272,003808. El cuantil normal utiliza aritmética aproximada; la etiqueta `closedForm` describe el método, no una cantidad decimal exacta ni una garantía para demanda no normal.

8.5. Dimensionamiento de lotes

El inventario final satisface $I_t = I_{t-1} + q_t - d_t$. QSB minimiza

$$\sum_t \left(S_t \mathbf{1}_{q_t > 0} + c_t q_t + h_t \operatorname{máx}(I_t, 0) + b_t \operatorname{máx}(-I_t, 0) \right).$$

La programación dinámica usa demanda y cantidades enteras, inventario inicial cero y final cero, permitiendo atraso con costo. No representa capacidad de producción ni stock inicial configurable. La exactitud se refiere al espacio de estados acotado de esta implementación educativa: inventarios entre menos y más la demanda total, y producción por período entre cero y esa demanda. El costo computacional crece con la demanda total y el horizonte. `prob_lotes.json` tiene tres demandas de diez unidades, preparación 20, producción unitaria 2, mantenimiento 1 y atraso 10. QSB devuelve producción $(20, 0, 10)$, inventario final $(10, 0, 0)$ y costo 110. La política $(10, 20, 0)$ también cuesta 110; un resultado óptimo no implica unicidad. La tabla de decisiones permite comprobar el balance y la imputación de costos a cada período.

8.6. Revisión con demanda incierta

Las cuatro políticas usan demanda normal por período y plazo constante. Con incremento independiente de demanda, el período de protección T tiene media DT y desviación $\sigma\sqrt{T}$. El stock de seguridad se expresa mediante el cuantil normal y el costo esperado de faltantes usa la función de pérdida normal. Son aproximaciones de cantidades continuas, marcadas

`approximate`; no resuelven una política óptima general para cualquier distribución o plazo aleatorio.

Los cuatro ejemplos comparten demanda anual 1200, desviación 100, plazo 0,02 años, preparación 30, mantenimiento 2, costo de atraso 20 y adquisición 5. Todo faltante queda pendiente; el costo de revisión es 5. El archivo de revisión continua hasta un nivel utiliza un tamaño medio de pedido suministrado de 200.

Archivo bajo `manual/examples/`	Política	Resultado principal
`prob_qr.json`	Cantidad fija (Q, r)	$Q = 194{,}866241, r = 54{,}242691$
`prob_sS.json`	Continua hasta nivel	$r = 54{,}095104, S = 254{,}095104$
`prob_periodica.json`	Intervalo fijo	$T = 0{,}170783, S = 321{,}459989$
`prob_opcional.json`	Periódica opcional	$T = 0{,}064550, s = 163{,}951339$

La revisión opcional devuelve además $S = 353{,}687998$. En continua fija, el stock de seguridad es 30,242691, el servicio 0,983761 y el costo relevante 450,217864. En periódica fija, la protección incluye revisión más plazo, 0,190783 años: confundir ambos intervalos reduce artificialmente el stock necesario. Las fracciones de atraso y venta perdida, sus costos y las distribuciones se validan en el núcleo.

8.7. Procedimiento en macOS y trazabilidad

Abra el JSON normalizado o seleccione una variante de inventarios en el creador de modelos. Complete demanda, costos y los parámetros específicos de la variante; valide antes de ejecutar. Mantenga visibles las unidades y revise las advertencias. En los resultados, EOQ muestra costos y comparación con el lote suministrado; descuentos muestra candidatos; vendedor de periódicos muestra servicio y resultados de demanda; lotes muestra cantidades y costos por período; revisión estocástica muestra niveles, stock de seguridad y costos. El selector de análisis/JSON conserva el documento del núcleo para exportación sin transcribir números de los gráficos. Las cantidades y fórmulas proceden de QSBCore, tanto en CLI como en macOS.

Todos los archivos de este capítulo se validan con `validate-inventory-json` y se resuelven con `solve-inventory-json`; añadir `-backend validate` valida sin resolver. El verificador `manual/verify_examples.py` comprueba los resultados esperados de `manual/examples/manifest.json`; no necesita material de referencia propietario.

8.8. Ejercicios

1. Compare pedidos enteros de 189 y 190 en EOQ y explique por qué conviene comparar costos en vez de redondear a ciegas.

2. Duplique el plazo del ejemplo estocástico: compare cambios en demanda protegida y stock de seguridad. ¿Se duplican ambos?

3. Baje el costo de atraso del ejemplo de lotes y compruebe si aparecen inventarios negativos. Interprete el atraso operacionalmente.

Capítulo 9

Calendarización y secuenciación

9.1. Problema y supuestos

Calendarizar es asignar operaciones a máquinas y tiempos. QSB minimiza el *makespan*, el instante en que termina la última operación [Pinedo, 2016]. Las operaciones no se interrumpen, una máquina atiende una operación a la vez y cada trabajo sigue su ruta. Las duraciones son enteras en la unidad indicada. No se incluyen tiempos de preparación, calendarios laborales ni fallos aleatorios. Los campos de fecha debida y peso se conservan, pero el objetivo nativo sigue siendo makespan; no optimiza tardanza ponderada.

9.2. Flow shop: una ruta común

En flow shop todos los trabajos pasan por las máquinas en el mismo orden. El solver nativo busca una permutación común a todas las máquinas. Para una secuencia dada, una operación empieza cuando están disponibles el trabajo y la máquina; su fin es inicio más duración. Respeta disponibilidades iniciales si se proporcionan.

En `manual/examples/plan_flowShop.json` hay dos trabajos y dos máquinas, inicialmente disponibles en cero:

Trabajo	M1	M2
J1	2	3
J2	3	1

```
swift run qsb validate-scheduling-json manual/examples/plan_flowShop.json
swift run qsb solve-scheduling-json manual/examples/plan_flowShop.json --backend
    native
```

La secuencia J1, J2 da makespan 6. En M1, J1 ocupa $[0, 2]$ y J2 $[2, 5]$; en M2, J1 ocupa $[2, 5]$ y J2 $[5, 6]$. La alternativa J2, J1 termina en 8. El ocio inicial de M2 no es una operación perdida: espera la salida de J1. La búsqueda es exacta dentro del modelo de permutación, con límite de ocho trabajos y costo factorial; no prueba optimalidad entre todas las posibles secuencias diferentes por máquina de un flow shop general.

9.3. Job shop: rutas propias

En job shop cada trabajo puede seguir una ruta distinta. El ejemplo `plan_jobShop.json` requiere J1: M1(2), M2(3), y J2: M2(2), M1(1). Las precedencias se deducen del orden de las operaciones de cada trabajo.

```
swift run qsb validate-scheduling-json manual/examples/plan_jobShop.json
swift run qsb solve-scheduling-json manual/examples/plan_jobShop.json --backend
    native
```

QSB obtiene makespan 5: J1 usa M1 de 0 a 2 y M2 de 2 a 5; J2 usa M2 de 0 a 2 y M1 de 2 a 3. La duración total de J1 es 5 y constituye una cota inferior, por lo que el calendario alcanza el óptimo. En este caso `jobSequence` refleja el orden de despacho de operaciones y puede repetir nombres; no debe interpretarse como una permutación de trabajos. La ramificación y acotación con poda de dominancia admite hasta seis trabajos y 36 operaciones. Son límites educativos: el tiempo de búsqueda puede crecer mucho antes de alcanzarlos. Native es determinista y marca ambas variantes `fixtureScale`; Validate no construye calendarios y External no está disponible.

9.4. Leer el Gantt y usar macOS

Importe el sobre JSON, cuyo `kind` es `flowShop` o `jobShop`. Para introducir o cambiar operaciones utilice el formulario estructurado de Definition o el JSON del modelo. Estas variantes no utilizan el editor de actividades de CPM/PERT. Son problemas distintos y los datos de uno no sustituyen al otro. Valide y ejecute con QSB Native. La solución abre Timeline: cada fila es una máquina, las barras son operaciones y los huecos muestran ocio. La escala horizontal permite revisar calendarios largos. El detalle de una operación identifica trabajo, máquina, inicio, fin y duración; JSON contiene también `idleBefore` y los calendarios por máquina.

Antes de aceptar el resultado, compruebe que ninguna máquina tiene barras solapadas, que cada trabajo conserva su orden y que el máximo fin coincide con makespan. Un calendario corto no garantiza cumplimiento de fechas debidas; compare estas por separado si forman parte de la decisión real. Los algoritmos trabajan en QSBCore y el Gantt presenta los mismos tiempos exportados por la CLI.

9.5. Ejercicios

1. Cambie la segunda operación de J2 en flow shop de 1 a 5. Compare ambas permutaciones y verifique el resultado.

2. En job shop añada una disponibilidad inicial de M2 igual a 2. Dibuje las esperas y compruebe que el solver no programa antes de ella.

3. Ponga una fecha debida de 4 a J1. Explique por qué cambiar ese campo no convierte el problema en minimización de tardanza.

Capítulo 10

Proyectos y planificación de producción

10.1. CPM: precedencias y camino crítico

CPM trabaja sobre una red acíclica de actividades: una actividad solo empieza cuando han terminado todas sus predecesoras. No asigna recursos limitados. El recorrido hacia delante calcula inicios y fines tempranos; el recorrido hacia atrás calcula los tardíos. La holgura total es inicio tardío menos inicio temprano. Una actividad crítica tiene holgura cero.

En `manual/examples/plan_cpm.json`, A dura 3 días, B dura 2 y ambas preceden a C, que dura 4. Cada actividad tiene costo normal 10, tiempo de crash una unidad menor y costo de crash 15.

```
swift run qsb validate-project-json manual/examples/plan_cpm.json
swift run qsb solve-project-json manual/examples/plan_cpm.json --backend native
swift run qsb solve-cpm-crash-json manual/examples/plan_cpm.json 6 --backend native
```

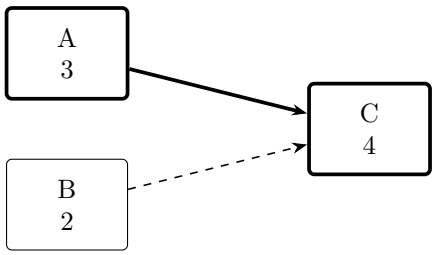


Figura 10.1: Red CPM de actividades: el número bajo cada nombre es su duración en días. El trazo grueso identifica actividades y precedencias críticas; el discontinuo muestra la otra precedencia.

El proyecto normal termina en 7 días. A y C son críticas; B tiene holgura 1. El costo normal total es 30. Crash para plazo 6 reduce C de 4 a 3, con costo incremental 5 y costo total 35. Reducir A también podría alcanzar el mismo costo; no suponga unicidad del plan.

La optimización de crash supone costos lineales entre tiempos normal y mínimo, con pendiente $(C_{crash} - C_{normal})/(t_{normal} - t_{crash})$ cuando hay reducción posible. Usa tiempos continuos y un LP educativo que considera caminos; el número de caminos puede crecer con rapidez. No es una política de contratación de recursos ni incluye costo indirecto del proyecto. Los plazos inferiores a lo alcanzable deben tratarse como infactibles, nunca como instrucciones para violar precedencias.

10.2. PERT: duración esperada e incertidumbre

PERT utiliza estimaciones optimista a, más probable m y pesimista b, ordenadas $a \leq m \leq b$. La aproximación clásica calcula

$$E[T] = (a + 4m + b)/6, \qquad \mathrm{Var}(T) = ((b - a)/6)^2.$$

En `plan_pert.json` las ternas son A(2,3,4), B(1,2,3) y C(3,4,5), con las mismas precedencias del ejemplo CPM.

```
swift run qsb solve-project-json manual/examples/plan_pert.json --backend native
```

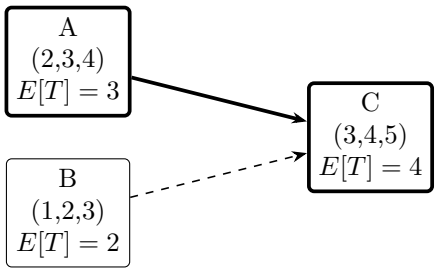


Figura 10.2: Red PERT: cada actividad muestra la terna optimista, más probable y pesimista, y su duración esperada en días. El trazo grueso identifica el camino crítico calculado con esas medias.

La programación de tiempos esperados vuelve a durar 7, con A y C críticas. La varianza reportada es 2/9 y la desviación estándar 0,471405 días. En este ejemplo hay un único camino crítico y la suma de sus varianzas presupone independencia. La implementación suma las varianzas de todas las actividades marcadas críticas; si hay varios caminos críticos, ese total no es la varianza de un único camino ni la del máximo entre ellos. El camino crítico puede cambiar entre realizaciones; sustituir duraciones aleatorias por sus medias no calcula exactamente la esperanza del máximo. La etiqueta `exact` describe el cálculo sobre esos tiempos esperados, no una predicción probabilística exacta de la fecha final. QSB no devuelve una probabilidad de cumplimiento ni simula esa incertidumbre en este módulo.

10.2.1. CPM y PERT en macOS

Importe el sobre de proyecto o cree una definición CPM/PERT. El editor de actividades recoge nombres, predecesoras y tiempos; CPM añade costos y PERT las tres estimaciones. Predecesoras inexistentes y ciclos aparecen en Validation. Cambiar de método conserva los datos de ambos métodos después de su inicialización: compruebe las estimaciones al volver a PERT. Copy Table/Paste Table permite trasladar actividades desde una hoja de cálculo. Run produce duración, actividades críticas, barras tempranas/tardías, holguras y tabla temporal. Revise JSON para la varianza y los valores exactos. La operación CLI de crash es un flujo separado; la vista normal no implica que se haya optimizado un plazo. Native y Validate están disponibles; External no resuelve proyectos actualmente.

10.3. Planificación agregada

La planificación agregada equilibra demanda por períodos mediante producción regular, horas extra, subcontratación, inventario y pedidos pendientes. Cuando la capacidad depende de plantilla, añade contratación y despido. Un balance básico es

$$I_t - B_t = I_{t-1} - B_{t-1} + R_t + O_t + S_t - D_t.$$

El objetivo suma costos de producción, capacidad ociosa, personal, inventario y atraso según los datos. QSB normaliza los métodos históricos simple, transporte y LP a una misma formulación LP continua; el campo `method` conserva su identidad de entrada, no selecciona tres algoritmos nativos distintos.

El ejemplo `plan_aggregate.json` tiene dos meses, demanda 4 y 6, capacidad regular 5 cada mes, costo regular 2 por unidad, horas extra hasta 2 con costo 5, inventario con costo 1 y sin atraso ni subcontratación. No depende de plantilla y empieza sin inventario.

```
swift run qsb validate-aggregate-json manual/examples/plan_aggregate.json
swift run qsb solve-aggregate-json manual/examples/plan_aggregate.json --backend
    native
```

El plan fabrica 5 en cada mes; conserva 1 al final del primero y cero al final del segundo. Cuesta $10 \cdot 2 + 1 = 21$. Fabricar exactamente la demanda del primer mes y usar una unidad extra en el segundo costaría $9 \cdot 2 + 5 = 23$. Las unidades deben ser consistentes: capacidad por trabajador no es capacidad total y `capacityRequirementPerUnit` convierte producción a capacidad. Los vectores deben tener el mismo número de períodos; `null` en máximos opcionales representa ausencia de cota. La plantilla y las cantidades son continuas, incluso si el negocio necesita personas enteras. No redondee una solución sin comprobar de nuevo balances y capacidad.

En macOS el editor de períodos separa demanda, costos, inventario/fuentes y plantilla. Copy Table transfiere las 20 columnas de períodos; Paste Table valida el rango antes de sustituirlo. Run muestra producción apilada y demanda, inventario, atrasos y tabla del plan. Un atraso aparece bajo cero en el gráfico, pero su cantidad del modelo es no negativa. Native resuelve el LP educativo; External usa HiGHS para la misma formulación si está instalado. Validate no produce un plan. El objetivo óptimo corresponde a los costos y cotas introducidos, no a criterios ausentes como estabilidad social de la plantilla.

10.4. MRP: explosión de necesidades

MRP parte de un programa maestro de producción (MPS), una lista de materiales (BOM), inventario, recepciones programadas y plazos. Recorre padres antes que componentes: la liberación de una orden del padre genera necesidades brutas para sus componentes, multiplicadas por el uso unitario [Silver et al., 1998]. Para un artículo con stock de seguridad SS, necesidad bruta G_t y recepción programada SR_t, la necesidad neta es $\max(0, G_t + SS - I_{t-1} - SR_t)$. Una recepción planificada cubre esa necesidad, y su liberación se desplaza hacia atrás por el plazo.

`plan_mrp.json` tiene semanas S0–S3. Cinco unidades de P deben recibirse en S3; P utiliza dos C por unidad. Ambos tienen plazo de una semana, stock inicial y seguridad cero, regla LFL y capacidad ilimitada.

```
swift run qsb validate-mrp-json manual/examples/plan_mrp.json
swift run qsb solve-mrp-json manual/examples/plan_mrp.json --backend native
```

P recibe 5 en S3 y libera 5 en S2. C necesita 10 en S2, recibe 10 entonces y libera 10 en S1. Los inventarios proyectados quedan en cero. Las necesidades de C provienen de la liberación de P, no de la fecha de demanda final. La BOM debe ser acíclica y todos sus identificadores deben existir.

Las reglas disponibles son LFL, EOQ, LUC, LTC y PPB. LFL recibe la necesidad neta; EOQ parte de $\sqrt{2DS/H}$ y recibe al menos la necesidad actual. En EOQ, `lotSizingLookahead` se interpreta como múltiplo de lote $m = \max(1, \text{valor configurado})$, no como número de períodos: la cantidad es $\max(\text{necesidad}, m\lceil EOQ/m \rceil)$. Si el costo de posesión anual es cero, recibe la necesidad neta. LUC compara costo unitario; LTC y PPB usan actualmente el mismo selector, que minimiza la diferencia absoluta entre costo de posesión y costo de pedido dentro

del horizonte configurado. Son reglas hacia delante, no una optimización global de costos multinivel. QSB marca el documento MRP como `heuristic`, incluso cuando el caso particular usa LFL. La capacidad finita genera `capacityExcess`; no nivela ni reprograma órdenes. Si la liberación cae antes del horizonte, QSB la acumula en el primer bucket mediante $\max(0, t - L)$; ese registro requiere revisión operativa y no prueba que el plazo pueda cumplirse.

En macOS Item Master recoge parámetros, Schedules cantidades por período y Bill of Materials relaciones padre/componente. Seleccione el artículo para editar o leer su calendario; los horizontes largos se agrupan para navegar. Copy/Paste Table siempre intercambia el rango completo de su sección. La solución muestra necesidades, inventario, recepciones, liberaciones y excesos de capacidad por artículo, con JSON como auditoría. Native y Validate están disponibles; External no está implementado para MRP.

10.5. Ejercicios

1. En CPM aumente B de 2 a 4. Recalcule camino crítico y holguras.

2. En PERT aumente solo el pesimista de A. Compare duración y varianza; explique qué supuestos limitan una interpretación probabilística.

3. Eleve el costo de inventario agregado de 1 a 4. Compare producir anticipadamente con usar horas extra.

4. Añada inventario inicial 4 de C en MRP y compruebe su necesidad neta. Después limite la capacidad de C en S2 a 5 y explique el exceso.

Capítulo 11

Instalaciones y equilibrado de líneas

11.1. Localización de una instalación

La localización continua minimiza distancia ponderada a puntos existentes:

$$\min_{x,y} \sum_i w_i d((x,y),(x_i,y_i)).$$

Los pesos representan interacciones por costo unitario, no probabilidades. QSB admite una instalación nueva y minimización; no resuelve selección discreta de almacenes, múltiples nuevas instalaciones ni problemas con barreras geográficas.

La medida de distancia determina el algoritmo. Para rectilínea, $d = |x-x_i|+|y-y_i|$, se usan medianas ponderadas por coordenada. Para euclídea al cuadrado, $d = (x-x_i)^2 + (y-y_i)^2$, se usa el centroide ponderado. Para euclídea ordinaria se usa iteración de Weiszfeld y el resultado es aproximado. No cambie la medida solo para obtener coordenadas visualmente más atractivas: cambia el objetivo y sus unidades.

`manual/examples/plan_location.json` coloca F1 en $(0,0)$ con peso 1 y F2 en $(4,0)$ con peso 3, y utiliza distancia euclídea al cuadrado. La matriz dirigida tiene esos valores desde las instalaciones existentes hacia la nueva; no hay otras interacciones. El peso efectivo suma ambos sentidos de interacción entre cada punto existente y el nuevo: registrar el mismo flujo en los dos sentidos duplica el peso.

```
swift run qsb validate-facilities-json manual/examples/plan_location.json
swift run qsb solve-facilities-json manual/examples/plan_location.json --backend
    native
```

El centro es $(3,0)$ y el objetivo $1\cdot 9+3\cdot 1=12$. En macOS Definition permite editar roles, IDs, coordenadas, distancia y matriz. Las coordenadas nuevas pueden quedar vacías; las existentes son necesarias. Los IDs se interpretan dentro del rol existente/nuevo, de modo que ambos pueden usar el ID 1. La solución representa puntos y conexiones ponderadas y detalla cada contribución. Use JSON para conservar la matriz completa.

11.2. Equilibrado de una línea

Con tareas indivisibles de tiempos t_i, precedencias y ciclo fijo C, se minimiza el número de estaciones. Una estación no puede exceder el ciclo y la estación de una predecesora no puede quedar después de su sucesora; tareas precedentes pueden compartir estación si se ejecutan en orden. La cota inferior $\lceil \sum_i t_i/C \rceil$ ignora incompatibilidades y precedencias, por lo que no siempre es alcanzable.

`plan_line.json` contiene A(2), B(3), C(4), con A y B precediendo a C, y ciclo 5 minutos.

```
swift run qsb validate-facilities-json manual/examples/plan_line.json
swift run qsb solve-facilities-json manual/examples/plan_line.json --backend native
```

Se asignan A y B a la estación 1 y C a la 2. Hay 9 minutos de trabajo y 10 minutos de capacidad por ciclo: eficiencia $9/(2 \cdot 5) = 0{,}9$ y retraso de balance $1 - 0{,}9 = 0{,}1$. El ocio es 0 y 1 minuto respectivamente. La cota inferior es 2 y se alcanza, probando optimalidad para el modelo. La programación dinámica exacta con máscaras admite como máximo 24 tareas; una tarea mayor que el ciclo hace el problema inviable. El campo `isolated` se conserva por compatibilidad, pero el solver nativo actual no obliga a una estación separada para las tareas marcadas. No lo use como una restricción efectiva de seguridad o incompatibilidad.

El editor macOS admite tareas, tiempos, sucesores inmediatos y flags; Copy/Paste Table transfiere la tabla. Borrar una tarea referenciada deja visible la referencia para repararla. La solución muestra carga y ocio por estación, asignaciones, contexto de precedencias, eficiencia y retraso. La eficiencia no mide tiempo total de producción de un lote ni incluye averías o cambios de producto.

11.3. Distribución en planta

Layout evalúa departamentos situados en una cuadrícula. Cada departamento ocupa la unión de uno o varios rectángulos de celdas; una celda repetida del mismo departamento cuenta una vez. Las coordenadas de rectángulos son inclusivas y empiezan en 1. El objetivo de carga–distancia usa centroides:

$$Z = \sum_{i,j} w_{ij}(|r_i - r_j| + |c_i - c_j|).$$

La matriz es dirigida: introducir el mismo peso en ambos sentidos duplica la contribución. El centroide es una aproximación de los recorridos físicos; no modela puertas, pasillos obligatorios ni obstáculos.

El ejemplo `plan_layout.json` tiene una fila y tres celdas A, B, C, todas de área uno y móviles. Sus únicos pesos son A–B=1, A–C=10 y B–C=1.

```
swift run qsb validate-facilities-json manual/examples/plan_layout.json
swift run qsb solve-facilities-json manual/examples/plan_layout.json --backend
    native
swift run qsb solve-facilities-json manual/examples/plan_layout.json \
  --backend native --layout-strategy pairwise-swap
```

La evaluación Initial da $1 + 20 + 1 = 22$ y no modifica la disposición. Pairwise intercambia A y B: B, A, C cuesta $1 + 10 + 2 = 13$, mejora 9. La búsqueda solo intercambia departamentos móviles de igual área cuando mejora el objetivo. Termina en un óptimo local de ese vecindario; no garantiza el óptimo global ni construye automáticamente geometría arbitraria. Las áreas fijas no se intercambian. Solapamientos móviles son errores; solapamientos con áreas fijas pueden producir advertencias y deben revisarse.

En Definition se editan cuadrícula, departamentos, flags fijos, rectángulos y matriz de flujos/costos. Seleccione un departamento para sus rectángulos; la tabla transferida incluye IDs para todos ellos. En matrices grandes se seleccionan fuentes y grupos de destinos, aunque Copy/Paste Table mueve la matriz completa. Elija Initial o Pairwise antes de Run. La vista Layout permite ajustar escala, cuadrícula y flujos, y revisar centroides, intercambios y contribuciones. Para cuadrículas grandes la vista general dibuja menos líneas; los rangos exactos siguen en detalle y JSON.

11.4. Backend, validación y decisión

Las tres familias admiten Native y Validate, sin backend External operativo. Validate informa datos inválidos sin calcular una localización, distribución o balance. Los documentos conservan algoritmo y exactitud: localización por centroide/medianas es de forma cerrada, Weiszfeld es aproximado, equilibrado es exacto a escala educativa y Pairwise es heurístico. Evaluar un layout inicial solo determina su costo, no su optimalidad. Los ejemplos usan el sobre facilities con `kind` igual a `location`, `lineBalancing` o `layout`. No confunda ese sobre con los JSON particulares aceptados por comandos específicos de cada subfamilia. Valide el mismo archivo que va a resolver y exporte el modelo y la solución para dejar trazabilidad de los datos aceptados.

11.5. Ejercicios

1. Cambie la localización a distancia rectilínea. Explique la nueva coordenada con medianas ponderadas y las nuevas unidades del objetivo.

2. En la línea reduzca el ciclo a 4. Determine el mínimo de estaciones y compruebe el impacto sobre eficiencia.

3. Fije A en layout. Resuelva Initial y Pairwise y compare los movimientos permitidos. Explique por qué una búsqueda sin mejora no demuestra optimalidad entre todas las distribuciones posibles.

Parte IV

Incertidumbre y calidad

Capítulo 12

Modelos de colas

12.1. Del flujo a la espera

La teoría de colas relaciona variabilidad, capacidad y congestión [Gross et al., 2008]. En la notación Kendall, M denota llegadas Poisson o tiempos exponenciales, según su posición. La tasa de servicio μ es por servidor; el tiempo medio es $1/\mu$. Todas las tasas comparten una unidad temporal. QSB implementa M/M/1 y una cola finita con varios servidores mediante un modelo de nacimiento y muerte.

12.2. M/M/1 en régimen estacionario

Se suponen llegadas independientes, servicio exponencial, un servidor, población y espera sin límite. La estabilidad exige $\lambda < \mu$. Con $\rho = \lambda/\mu$:

$$P_0 = 1 - \rho, \quad L = \frac{\rho}{1-\rho}, \quad L_q = \frac{\rho^2}{1-\rho}, \quad W = \frac{1}{\mu - \lambda}, \quad W_q = \frac{\lambda}{\mu(\mu - \lambda)}.$$

L incluye clientes atendidos; L_q solo quienes esperan. La ley de Little proporciona $L = \lambda W$ y $L_q = \lambda W_q$. El backend declara `mm1ClosedForm`/`closedForm`. No calcula la distribución de un tiempo de espera individual ni percentiles.

12.2.1. Ejemplo: una ventanilla

`manual/examples/prob_cola.json` especifica cuatro llegadas/hora y seis servicios/hora. Ejecute desde la raíz:

```
.build/debug/qsb validate-queuing-json manual/examples/prob_cola.json
.build/debug/qsb solve-queuing-json manual/examples/prob_cola.json
```

La utilización es 2/3, $P_0 = 1/3$, $L = 2$, $L_q = 4/3$, $W = 0{,}5$ horas y $W_q = 1/3$ horas: treinta minutos en el sistema, veinte esperando y diez de servicio. Estar ocupado dos tercios del tiempo no significa que dos tercios de los clientes esperen veinte minutos exactamente. Si la llegada alcanza seis/hora, el modelo deja de tener estas medias estacionarias y QSB devuelve un diagnóstico de estabilidad.

12.3. Varios servidores y capacidad finita

Con c servidores y b plazas de espera, la capacidad del sistema es $K = c + b$. El estado n tiene tasa de llegada λ si $n < K$ y cero cuando está lleno; su tasa de salida es $\min(n, c)\mu$. Se

normalizan las probabilidades obtenidas por

$$p_n = p_0 \prod_{i=1}^{n} \frac{\lambda}{\text{mín}(i, c)\mu}.$$

La probabilidad de bloqueo es p_K, la llegada efectiva $\lambda_e = \lambda(1 - p_K)$, y Little utiliza λ_e, no la tasa ofrecida. Al ser finito, este modelo puede tener distribución estacionaria incluso con demanda ofrecida superior a su capacidad de servicio; ello produce bloqueo, no capacidad adicional.

QSB requiere llegadas exponenciales en lotes de una unidad y población no finita. Usa el tiempo medio de servicio para la tasa por servidor; las etiquetas no exponenciales generan advertencias porque su variabilidad no está modelada. El documento conserva siempre `approximate` para esta ruta. No debe presentarse un caso normal como solución exacta M/G/c/K.

12.3.1. Ejemplo: dos ventanillas y tres plazas

`prob_cola_finita.json` conserva cuatro llegadas/hora y diez minutos medios de servicio, con dos servidores y tres plazas de espera. Valide y resuelva con los mismos comandos cambiando el archivo. Se obtiene:

Métrica	Valor
Capacidad total	5
Probabilidad de bloqueo	0.004124
Llegadas efectivas por hora	3.983505
Utilización media por servidor	0.331959
Número medio en el sistema	0.738144
Número medio esperando	0.074227
Tiempo en el sistema (horas)	0.185300
Tiempo esperando (horas)	0.018634

La espera media es aproximadamente 1,118012 minutos entre clientes admitidos. La utilización total de ambos servidores equivale a 0,663918 servidores ocupados, no al porcentaje individual.

12.4. Costos, interfaz y límites

Los costos opcionales distinguen servidor ocupado y ocioso, clientes esperando y en servicio; la cola finita añade rechazo y plazas. Los costos de estados se ponderan por ocupación esperada; el rechazo se multiplica por la tasa de clientes bloqueados. El total es un costo por unidad temporal, no por cliente.

En macOS, el creador ofrece ambas variantes. Introduzca tasas para M/M/1 o tiempos medios, servidores y espacio de espera para cola finita; valide y resuelva. El panel de métricas muestra utilización, flujo efectivo, bloqueo, números y tiempos. La variante finita muestra además probabilidades de estado con el estado lleno distinguido; los costos opcionales aparecen desglosados. Compruebe la unidad temporal y las hipótesis junto a los resultados. El JSON normalizado es la alternativa de consulta y exportación.

Use `solve-queuing-json` para recibir una entrada normalizada.

Los comandos históricos `solve-mm1-json` y `solve-finite-queue-json` reciben un archivo legado, pese a su sufijo: este señala la salida JSON. Para intercambiar entradas normalizadas use siempre la ruta genérica. Los backends son nativo y validación; no hay solver externo instalado para esta familia.

No se modelan prioridades, abandono por paciencia, clientes con población finita, llegadas por lotes generales o variación de tasas por hora del día. La simulación del capítulo de Markov y simulación permite explorar otra clase de supuestos, pero sus estimaciones no sustituyen estas fórmulas estacionarias.

12.5. Ejercicios

1. Cambie λ a 5.5 en M/M/1 y compare utilización y espera. Explique por qué el crecimiento de la espera no es lineal.

2. Elimine las plazas de espera de la cola finita. Compruebe que hay bloqueo sin una cola positiva y use λ_e para verificar Little.

3. Compare añadir un servidor con añadir cinco plazas: ¿cuál cambia la capacidad de servicio y cuál cambia el almacenamiento de clientes?

Capítulo 13

Pronósticos y regresión

13.1. Una previsión es una hipótesis operativa

Los pronósticos alimentan inventarios, capacidad y planificación. Una recta que ajusta bien el pasado no garantiza precisión futura [Hyndman and Athanasopoulos, 2021]. QSB implementa tendencia lineal, media móvil, suavizamiento exponencial simple, descomposición estacional multiplicativa y regresión lineal por mínimos cuadrados. El JSON es una solicitud: contiene modelo, método, horizonte y parámetros. Así se conserva la elección del método junto con los datos. Los backends nativo y de validación están disponibles; la integración externa es una extensión futura.

13.2. Datos y errores

Las observaciones se procesan en el orden de sus filas; las etiquetas no reordenan fechas ni completan períodos ausentes. Una serie mensual debe tener espaciado mensual coherente. QSB devuelve valores ajustados, residuos $e_t = y_t - \hat{y}_t$, MAD, MSE y MAPE:

$$\mathrm{MAD} = \frac{1}{m}\sum |e_t|, \quad \mathrm{MSE} = \frac{1}{m}\sum e_t^2, \quad \mathrm{MAPE} = \frac{100}{m_+}\sum_{|y_t|>10^{-12}} |e_t/y_t|.$$

m cuenta las filas ajustadas disponibles, no necesariamente toda la historia; m_+ excluye valores reales con magnitud no mayor que 10^{-12}. Si todos quedan excluidos, MAPE no está disponible. MAD conserva unidades de la serie; MSE tiene unidades al cuadrado; MAPE se publica en porcentaje. Comparar métodos exige considerar que media móvil y suavizamiento omiten filas iniciales diferentes.

13.3. Tendencia lineal: ejemplo completo

El modelo $y_t = a + bt + e_t$ estima a, b por mínimos cuadrados con índices $t = 1, \ldots, n$. `manual/examples/prob_pronostico.json` contiene $(12, 14, 16, 18, 20, 22)$, unidad mes y horizonte dos. Desde la raíz:

```
.build/debug/qsb validate-forecast-json manual/examples/prob_pronostico.json
.build/debug/qsb solve-forecast-json manual/examples/prob_pronostico.json
```

QSB obtiene $a = 10, b = 2$, pronósticos 24 y 26, y los tres errores cero. Son datos artificiales elegidos para comprobar fórmula e índices: el error nulo no constituye evidencia sobre demanda real. El resultado conserva la solicitud, el algoritmo `linearTrend` y exactitud `exact`, que describe el cálculo del método, no el conocimiento exacto del futuro.

13.4. Media móvil y suavizamiento

Para ventana k, la previsión de un paso es

$$\hat{y}_{t+1} = \frac{1}{k}\sum_{j=0}^{k-1} y_{t-j}.$$

QSB extiende horizontes usando previsiones anteriores dentro de la ventana cuando se agotan valores reales. El archivo `prob_movil.json` selecciona ventana tres sobre la misma serie. Resultan 20 y 20,666667: la segunda media utiliza $(20, 22, 20)$. MAD es 4, MSE 16 y MAPE 20,134680 %, calculados sobre las últimas tres filas. Esta demora es previsible cuando la serie tiene tendencia.

El suavizamiento simple aplica

$$F_{t+1} = \alpha y_t + (1-\alpha)F_t, \qquad 0 < \alpha \leq 1.$$

El pronóstico inicial se fija al primer dato. En `prob_suavizado.json`, $\alpha = 0{,}3$ produce 18,117660 en ambos períodos futuros. El método no tiene componente de tendencia; no extrapola automáticamente el crecimiento. Su MAD es 4,078440, MSE 18,233818 y MAPE 22,081810 %. Un alfa grande reacciona rápido y puede seguir más ruido; uno pequeño suaviza y puede retrasarse. Ambos ejemplos se ejecutan con `solve-forecast-json` y se validan con `validate-forecast-json`.

13.5. Estacionalidad multiplicativa

El método combina la tendencia estimada con factores de estación: $\hat{y}_t = (a + bt)s_j$. QSB calcula cocientes real/tendencia, promedia por posición estacional y normaliza los factores para que su media sea uno. Es una descomposición educativa, no Holt–Winters ni un modelo ARIMA. La longitud estacional y el orden de observaciones deben corresponder al fenómeno; se necesitan al menos dos ciclos completos y valores/tendencia compatibles con la división multiplicativa. `prob_estacional.json` usa longitud dos sobre la serie lineal anterior. Los dos factores son uno; la previsión vuelve a ser $24, 26$. El caso comprueba que especificar una estación no inventa oscilaciones cuando los datos coinciden con la recta. Para estudiar estacionalidad real, cambie la serie por varios ciclos con oscilación y revise residuos antes de interpretar los factores.

13.6. Regresión explicativa

Se modela $y_i = \beta_0 + \sum_j \beta_j x_{ij} + e_i$ con intercepto. Los predictores deben tener columnas linealmente independientes; una columna constante duplica el intercepto. El núcleo detecta deficiencia de rango. `prob_regresion.json` contiene actividad $1, 2, 3, 4, 5$ y consumo $5, 7, 9, 11, 13$. Con `solve-forecast-json`, QSB devuelve intercepto 3, coeficiente de actividad 2, SSE cero y $R^2 = 1$. El horizonte del contrato de solicitud debe ser positivo también para regresión, aunque su salida ajusta las observaciones suministradas y no genera filas futuras de predictores. Para una nueva actividad, la ecuación estimada permite calcular una predicción, sin que QSB publique un intervalo para ella.

Un coeficiente es una asociación condicional, no causalidad. R^2 mide ajuste dentro de la muestra; no identifica sobreajuste ni garantiza extrapolación. No se publican errores estándar de coeficientes, pruebas de hipótesis ni intervalos predictivos en este flujo. Datos originales pequeños permiten comprobar el contrato, pero no evaluar capacidad de generalización.

13.7. Trabajo en macOS y evaluación

En el editor de pronósticos seleccione método, parámetros y horizonte; introduzca filas históricas en el orden temporal. Se pueden pegar valores en una columna, o etiquetas y valores separados por tabuladores en dos columnas. El pegado se valida de forma atómica: una fila mal formada no reemplaza parcialmente la historia. La regresión utiliza nombres de variables y filas explicativas. Valide, resuelva y revise las curvas reales, ajustadas y futuras; las curvas y el gráfico de residuos comparten el orden de observaciones. Controles de visibilidad y escala horizontal ayudan con series extensas. El resumen muestra los parámetros y métricas; el selector JSON conserva solicitud y resultado. Aplicar JSON solo actualiza el borrador cuando su decodificación funciona.

Para evaluar un pronóstico operacional, reserve períodos finales, ajuste con la historia anterior y compare con datos reservados. Los errores publicados por QSB son los ajustes del cálculo ejecutado; no constituyen automáticamente una prueba fuera de muestra. No seleccione parámetros mirando reiteradamente el mismo conjunto reservado. Documente unidades, ventana, alfa, estación y horizonte al entregar el análisis.

13.8. Ejercicios

1. Compare ventanas dos y cuatro sobre la serie creciente. Explique el retraso y compruebe qué filas participan en MAD.

2. Sustituya un dato por cero y compruebe la definición de MAPE. Compare su conveniencia con MAD.

3. Introduzca un segundo predictor igual al doble del primero y observe el diagnóstico de rango. Explique por qué más columnas no siempre aportan información.

Capítulo 14

Procesos de Markov y simulación

14.1. Dos formas de describir la incertidumbre

Una cadena de Markov propaga probabilidades entre estados; una simulación construye realizaciones de eventos y estima medidas por repetición. La primera puede dar una distribución estacionaria mediante ecuaciones; la segunda admite redes y reglas más detalladas, a cambio de error de muestreo. QSB conserva modelos y resultados normalizados distintos para ambos procedimientos [Ross, 2019, Law, 2015].

14.2. Cadena finita y convención de filas

P_{ij} es la probabilidad de pasar del estado i al j en un período. Cada fila suma uno. La distribución es un vector fila:

$$s_{t+1} = s_t P, \qquad \pi P = \pi, \qquad \sum_i \pi_i = 1.$$

Se supone transición homogénea en el tiempo y que el próximo estado depende del actual, sin información adicional del pasado. Los costos c_i son por estado y período; su esperanza es $s_t c$. No son costos de transición.

14.2.1. Ejemplo: disponibilidad de una máquina

`manual/examples/prob_markov.json` define Operativa y Parada, costos $(0, 100)$, distribución inicial $(1, 0)$, horizonte tres y

$$P = \begin{pmatrix} 0{,}9 & 0{,}1 \\ 0{,}4 & 0{,}6 \end{pmatrix}.$$

Desde la raíz del repositorio:

```
.build/debug/qsb validate-markov-json manual/examples/prob_markov.json
.build/debug/qsb solve-markov-json manual/examples/prob_markov.json
```

Los resultados son:

Período	Operativa	Parada	Costo esperado
0	1	0	0
1	0.9	0.1	10
2	0.85	0.15	15
3	0.825	0.175	17.5

La distribución estacionaria es (0,8, 0,2), con costo esperado 20 por período. Es una media de largo plazo, no un costo acumulado de tres períodos. Esta cadena es irreducible y aperiódica, por lo que las distribuciones transitorias convergen a ella. En general, unicidad estacionaria y convergencia desde cualquier inicio no son equivalentes: una cadena periódica puede oscilar.

QSB resuelve un sistema lineal estacionario y propaga matrices; declara `exact` como carácter algorítmico, con aritmética de coma flotante. El método exige una distribución estacionaria única y devuelve error si su sistema es singular. No elige una mezcla arbitraria en cadenas con varias clases cerradas. Si falta distribución inicial, no inventa una: la salida transitoria está vacía. Validar filas y dimensiones no garantiza que el sistema estacionario sea resoluble.

En macOS, el editor permite nombres, costos, probabilidades iniciales opcionales, matriz de transición y horizonte. Verifique la orientación de filas antes de resolver; el resultado muestra estacionario, propagación, costos e hipótesis. La edición de estados mantiene dimensiones coherentes; el texto inválido permanece visible hasta corregirse. JSON sigue disponible para intercambio.

14.3. Simulación de eventos discretos

Una red contiene fuentes de entidades, colas, servidores y compuertas. Las rutas incluyen probabilidades y tiempos de transferencia; las reglas de servicio pueden depender del tipo de entidad. QSB admite distribuciones constante, exponencial, normal, uniforme y triangular. Las muestras normales negativas se recortan a cero; no se vuelve a muestrear. Ese recorte modifica la media y la variabilidad respecto de una normal sin recorte. Sus parámetros representan tiempos o valores de la distribución, no tasas por unidad temporal. Por ejemplo, una media exponencial 0,25 horas equivale a cuatro llegadas/hora.

El backend educativo modela colas FIFO finitas, servidores alternativos, enrutamiento probabilístico, transferencias y sincronización de ensamblaje. Sus salidas son estimaciones `approximate`, aunque una semilla fija las haga reproducibles. No hay borrado de calentamiento, traza interactiva de eventos ni garantía de estado estacionario. Tampoco hay backend externo de simulación disponible.

14.3.1. Ejemplo: una ventanilla simulada

`prob_simulacion.json` conecta fuente Cliente, cola Espera de veinte plazas y servidor Atención. Cada llegada trae una entidad, el tiempo entre llegadas es exponencial de media 0,25 horas, y el servicio exponencial medio es 1/6 horas. Las transferencias duran cero.

```
.build/debug/qsb validate-simulation-json manual/examples/prob_simulacion.json
.build/debug/qsb solve-simulation-json manual/examples/prob_simulacion.json
.build/debug/qsb solve-simulation-json manual/examples/prob_simulacion.json --
    replications 3
```

La ejecución predeterminada dura 1000 horas, semilla 1, sin calentamiento eliminado. Genera 4005 entidades, completa 4004, no rechaza ninguna, y estima longitud media de cola 1,202238, máximo 15, y utilización 0,664470. El cliente sin completar puede estar en cola o servicio al terminar: no es necesariamente rechazado.

Con tres réplicas independientemente sembradas, la longitud media es 1,229240, intervalo normal aproximado del 95 % [1,146668, 1,311813]; la utilización es 0,659950, intervalo [0,638179, 0,681722]. Los contadores agregados publicados son medias redondeadas: generan 4006 y completan 4005, no la suma de tres corridas. Los máximos, en cambio, conservan el máximo entre réplicas. La semilla base, horizonte y número de réplicas figuran en la salida. La CLI expone réplicas, pero no opciones para elegir semilla u horizonte en estos comandos; esos valores proceden de las opciones predeterminadas del núcleo.

La fórmula del intervalo usa $\bar{x} \pm 1{,}96s/\sqrt{R}$. Con pocas réplicas, la aproximación normal puede ser pobre; aumentar R reduce el error de muestreo, no corrige sesgo de arranque ni una distribución incorrecta. No interprete el intervalo como garantía de que el modelo describe el mundo. El M/M/1 analítico del capítulo de colas da $L_q = 4/3$ y $\rho = 2/3$. La simulación tiene cola finita y horizonte finito: la proximidad es útil para comprobar unidades, pero no exige igualdad exacta. Tres réplicas tampoco justifican deducir un fallo porque un intervalo no contenga el valor analítico.

14.4. Procedimiento en macOS

En la definición de simulación, complete Componentes, Rutas y Reglas de servicio. Introduzca los parámetros de cada distribución por separado. Los nombres deben coincidir con las referencias de rutas y tipos de entidad. Se pueden intercambiar tablas completas con hojas de cálculo; el pegado comprueba cada sección antes de reemplazarla. En el panel Run seleccione entre una y cien réplicas, valide y ejecute. La interfaz muestra estado de cálculo y evita ejecuciones duplicadas; cambiar el modelo invalida un resultado pendiente. Esto no detiene un cálculo ya iniciado en el núcleo.

El panel de resultados muestra entidades generadas/completadas, tasa de completado, horizonte, semilla, cola y utilización, con intervalos cuando están disponibles. Consulte tablas y JSON además de gráficos. Conserve red, reglas, semilla, horizonte y réplicas al comparar escenarios: cambiar cualquiera puede cambiar la interpretación de la diferencia.

14.5. Ejercicios

1. Quite la distribución inicial del caso Markov y compruebe que queda solo análisis estacionario. Cambie después la matriz a identidad y observe el error del sistema singular.

2. Duplique la media de servicio simulada y compare terminaciones, rechazo y utilización. Explique por qué capacidad de cola no equivale a capacidad de servicio.

3. Repita el caso con diez réplicas y compare amplitud de intervalos. Separe variabilidad entre corridas de diferencias entre los supuestos de modelos.

Capítulo 15

Control de calidad y muestreo de aceptación

15.1. Vigilar un proceso o aceptar un lote

El control estadístico busca señales de cambio en un proceso; el muestreo de aceptación decide sobre lotes mediante muestras. Un proceso estable puede producir defectos, y aceptar un lote no certifica que todas sus unidades sean conformes. Ambos procedimientos necesitan definiciones y muestreo coherentes [Montgomery, 2020].

QSB admite cartas c, p y $\bar{X}$-R, Pareto y gráfico de probabilidad normal; evalúa además planes de aceptación simples y dobles binomiales. El backend nativo es determinista y el de validación no resuelve. No hay backend externo disponible para estas familias.

15.2. Cartas c y p

La carta c cuenta defectos en una unidad de inspección de tamaño constante. Un artículo puede tener más de un defecto. Con hipótesis de conteo Poisson,

$$CL = \bar{c}, \qquad LCL = \max(0, \bar{c} - 3\sqrt{\bar{c}}), \quad UCL = \bar{c} + 3\sqrt{\bar{c}}.$$

`manual/examples/prob_calidad.json` contiene conteos $(2, 3, 4, 3, 2, 4, 3, 3, 2, 4)$. Desde la raíz:

```
.build/debug/qsb validate-quality-json manual/examples/prob_calidad.json
.build/debug/qsb solve-quality-json manual/examples/prob_calidad.json
```

La línea central es 3, límite inferior cero y superior 8,196152. No hay puntos fuera de límites ni señales de las reglas implementadas. Es una comprobación didáctica de fórmula; diez conteos no bastan para establecer un proceso industrial como estable. Unidades de inspección de tamaños variables requieren otro planteamiento; QSB no implementa una carta u en esta ruta.

La carta p utiliza proporción de unidades defectuosas, con tamaño n_i para cada subgrupo. Su centro ponderado es

$$\bar{p} = \frac{\sum_i n_i p_i}{\sum_i n_i}, \qquad CL_i = \bar{p}, \quad CL_i \pm 3\sqrt{\bar{p}(1-\bar{p})/n_i},$$

recortando límites a $[0, 1]$.

En `prob_p.json`, tamaños $(100, 100, 200)$ y proporciones $(0{,}02, 0{,}04, 0{,}03)$ producen $\bar{p} =$ 0,03; los límites superiores son 0,081176 para los dos primeros subgrupos y 0,066187 para el

tercero. El inferior es cero. La entrada usa fracciones: 0,03 significa 3 %, no 0,03 %. Los límites de tres sigmas son una aproximación estadística de la distribución de los datos, aunque QSB etiquete como `exact` la aplicación de fórmulas.

15.3. Carta de medias y rangos

Subgrupos racionales agrupan mediciones tomadas bajo condiciones comparables. Con $\bar{\bar{X}}$ media de medias y $\bar{R}$ media de rangos,

$$LCL_{\bar{X}}, UCL_{\bar{X}} = \bar{\bar{X}} \mp A_2\bar{R}, \qquad LCL_R = D_3\bar{R}, \quad UCL_R = D_4\bar{R}.$$

QSB usa constantes estándar para tamaños uniformes entre dos y diez. Revise primero la variación en R antes de interpretar desplazamientos de medias. `prob_xr.json` contiene subgrupos $(9, 10, 11)$, $(10, 11, 12)$ y $(8, 9, 10)$. Se obtiene $\bar{\bar{X}} = 10$, $\bar{R} = 2$, límites de medias [7,954, 12,046] y de rangos $[0, 5{,}148]$, sin puntos fuera de límites. Los límites de control describen variabilidad del proceso; no son tolerancias de diseño. QSB no calcula índices Cp/Cpk en este flujo.

15.4. Pareto y probabilidad normal

Pareto agrega categorías, las ordena por frecuencia descendente y publica participaciones y acumulados. `prob_pareto.json` tiene filas $(4, 2, 1)$ y $(2, 1, 0)$ para Rayas, Manchas y Roturas. Sus totales son $(6, 3, 1)$, total 10, participaciones (0,6, 0,3, 0,1) y acumulados $(0{,}6, 0{,}9, 1)$. Los campos llamados `percentage` en JSON almacenan fracciones, no valores entre cero y cien. Priorizar Rayas por frecuencia no establece que sea la categoría de mayor costo o riesgo.

El gráfico normal ordena mediciones y asigna posiciones de Blom $p_i = (i - 3/8)/(n + 1/4)$; utiliza $z_i = \Phi^{-1}(p_i)$, ajustando medición contra puntuación normal. `prob_normal.json` contiene $(8, 9, 10, 11, 12)$; media 10, desviación muestral 1,581139, intercepto 10, pendiente 1,742919 y correlación 0,997902. Se etiqueta `approximate` por el cuantil normal aproximado. Una correlación elevada con cinco puntos no demuestra normalidad; no es una prueba con p-valor. Estos cuatro ejemplos se ejecutan con los mismos comandos de calidad cambiando el archivo.

15.5. Señales e interfaz de calidad

QSB señala puntos fuera de tres sigmas y reglas fijas Western Electric: dos de tres más allá de dos sigmas del mismo lado, cuatro de cinco más allá de una sigma del mismo lado y ocho del mismo lado del centro. Los puntos fuera de tres sigmas ya señalados se excluyen del recuento de las reglas zonales de dos de tres y cuatro de cinco. No hay configuración de reglas ni metadatos normalizados de causa, acción y comentario. Los índices de señales identifican observaciones a investigar, no diagnósticos automáticos de causa. Los límites recortados de cartas de atributos afectan la escala usada por las reglas de esta implementación: conserve la salida y revise la señal junto al gráfico.

En macOS seleccione la variante en el editor nativo y complete su tabla: conteos/proporciones, subgrupos, categorías o mediciones. Cambiar variante conserva tablas no activas durante la sesión, pero el JSON exporta solamente la activa. Pegue tablas completas para reemplazarlas de forma atómica; datos incompletos importados permanecen como celdas inválidas. Tras validar y resolver, las cartas muestran centros, límites y señales; Pareto muestra frecuencia y acumulado; el gráfico normal muestra puntos y recta. Consulte las tablas y el JSON para los valores exactos del cálculo y mantenga el orden de adquisición.

15.6. Plan simple de aceptación

Para tamaño de muestra n y máximo aceptable de defectuosas c, se acepta si $X \le c$. Bajo muestreo binomial independiente con proporción p, la curva característica operativa es

$$P_a(p) = \sum_{x=0}^{c} \binom{n}{x} p^x (1-p)^{n-x}.$$

AQL es el nivel de calidad aceptable y RQL el rechazable. El riesgo real del productor es $\alpha = 1 - P_a(AQL)$; el del consumidor es $\beta = P_a(RQL)$. Introducir riesgos nominales no modifica un plan ya suministrado para que los cumpla.

`prob_muestreo.json` fija $n = 20, c = 1, AQL = 0{,}01, RQL = 0{,}15$, riesgos nominales $0{,}05, 0{,}10$ y lote de 500.

```
.build/debug/qsb validate-acceptance-json manual/examples/prob_muestreo.json
.build/debug/qsb solve-acceptance-json manual/examples/prob_muestreo.json
```

QSB devuelve $P_a(AQL) = 0{,}983141$, riesgo del productor 0,016859 y riesgo del consumidor 0,175558. El plan supera el riesgo nominal del consumidor: no debe describirse como un plan que satisface $\beta = 0{,}10$. La salida contiene 101 puntos de curva desde 0 a 100 % defectuoso, además de medidas en AQL y RQL. Esta malla no sustituye evaluación continua para optimizar un riesgo entre puntos.

El tamaño medio de muestra ASN es 20. Bajo inspección rectificadora de lotes rechazados y eliminación de defectuosas inspeccionadas,

$$ATI = n + (N-n)(1-P_a), \qquad AOQ = pP_a(N-n)/N.$$

En AQL resulta ATI 28,092482 y AOQ 0,009438, es decir 0,943815 %. Si los lotes rechazados se devuelven sin inspección total, estas medidas no describen el procedimiento real. QSB evalúa las probabilidades binomiales exactamente dentro del supuesto, pero muestrear sin reemplazo una fracción grande del lote requiere considerar la distribución hipergeométrica; el núcleo ofrece esa probabilidad, no un selector en este comando.

15.7. Plan doble y diseño

En un plan doble, se inspeccionan n_1 unidades. Si las defectuosas $x_1 \le c_1$, se acepta; si $x_1 \ge r_1$, se rechaza; en medio se inspeccionan n_2 *adicionales*. Se acepta finalmente si $x_1 + x_2 \le c_2$. Confundir n_2 con tamaño acumulado cambia el plan. Su probabilidad de aceptación es

$$P_a = P(X_1 \le c_1) + \sum_{x=c_1+1}^{r_1-1} P(X_1 = x) P(X_2 \le c_2 - x).$$

El archivo `prob_doble.json` usa $n_1 = 10, c_1 = 0, r_1 = 3$, $n_2 = 10, c_2 = 1$ con los mismos niveles y lote anteriores. Cambiando el archivo en `solve-acceptance-json`, se obtiene riesgo del productor 0,013001, consumidor 0,265274, ASN en AQL 10,955041 y en RQL 16,233221. Reduce inspección media a costa de aceptar más lotes malos en este ejemplo; no es una mejora universal. ATI en AQL es 17,313145 y AOQ 0,009654, bajo la convención rectificadora del solver. Para planes dobles, QSB sustituye n por ASN en las fórmulas publicadas de ATI y AOQ. Es una convención agregada: no calcula por separado el tamaño inspeccionado condicionado a cada ruta de aceptación o rechazo.

La CLI permite diseñar el menor plan *simple* que cumple cotas binomiales:

```
.build/debug/qsb design-acceptance 500 0.01 0.15 0.05 0.10
```

El resultado es $n = 25, c = 1$, riesgo productor 0,025759 y consumidor 0,093071. El orden es lote, AQL, RQL, riesgo productor y riesgo consumidor. El comando busca un plan, a diferencia de evaluar uno ya fijado. No diseña planes dobles, no optimiza costo esperado, no implementa errores no nulos de clasificación y no afirma conformidad con una norma de muestreo específica.

En macOS el formulario distingue plan simple/doble y sus límites de decisión; conserva los niveles, riesgos y datos económicos compartidos al cambiar tipo. Valide y resuelva para ver curva OC, marcadores AQL/RQL, riesgos reales frente a nominales y ASN/ATI/AOQ. Los campos económicos se preservan en el modelo; la salida del núcleo contiene las medidas de calidad e inspección, no un óptimo económico. El JSON permite auditar el plan exacto utilizado.

15.8. Ejercicios

1. Añada un conteo de veinte al ejemplo c y observe cómo cambian centro y límites al incluirlo en la estimación. Discuta cuándo fijar una fase de referencia.

2. Compare cartas p con tamaño cien y mil manteniendo proporciones. Explique por qué una misma desviación puede producir señales diferentes.

3. Diseñe un plan simple con las cotas del ejemplo y compare ASN con el plan $20, 1$. Compruebe riesgos reales, no solo nominales.

Apéndice A

Referencia de términos

Backend Implementación seleccionada para validar o resolver una familia. La opción de validación no calcula una solución.

Balance Igualdad que contabiliza entradas, salidas y cambios de estado, como inventario final igual a inicial más producción menos demanda.

Capacidad Límite de un recurso, con unidad y horizonte definidos.

Diagnóstico Mensaje de validación con gravedad, código y, cuando está disponible, ruta del campo afectado.

Estado Resumen de información suficiente para decisiones futuras en un modelo dinámico; también puede referirse a una condición aleatoria del entorno.

Exactitud Caracterización del procedimiento y sus garantías para el modelo admitido, distinta de la cantidad de decimales mostrados.

Factibilidad Cumplimiento de restricciones, límites y condiciones del modelo; no equivale a optimalidad.

Función objetivo Expresión que cuantifica el criterio a maximizar o minimizar.

Heurística Procedimiento que busca una buena solución sin garantizar necesariamente un óptimo global.

Holgura Capacidad o margen restante de una restricción para una solución.

JSON normalizado Representación estructurada de un modelo o resultado que permite intercambiar, inspeccionar y reproducir datos entre clientes de QSB.

Makespan Tiempo de finalización del último trabajo de un programa.

Óptimo global Solución factible con el mejor objetivo en toda la región factible; un óptimo local sólo se compara con soluciones próximas.

Parámetro Dato que el modelo toma como conocido durante una ejecución.

Política Regla o conjunto de decisiones por etapa y estado.

Posterior Probabilidad de un estado después de observar evidencia, calculada a partir de previas y verosimilitudes.

Precio sombra Cambio marginal del objetivo al variar un recurso en el intervalo y bajo las condiciones en que la interpretación dual es válida.

Región factible Conjunto de soluciones que satisfacen las restricciones.

Relajación lineal Modelo que sustituye restricciones de integralidad por variables continuas; su solución puede no ser admisible para el problema entero.

Restricción Condición que limita las decisiones admisibles.

Semilla Valor que inicializa el generador pseudoaleatorio; permite repetir una simulación bajo el mismo modelo, configuración e implementación.

Sensibilidad Análisis de cómo cambia la solución al modificar datos o supuestos.

Solución óptima Solución factible que alcanza el mejor objetivo del problema considerado, dentro de las garantías y tolerancias del método.

Tolerancia Umbral numérico usado para comparar cantidades o aceptar aproximaciones; debe interpretarse junto a la escala del problema.

Traza Registro de etapas, estados y decisiones que facilita examinar una política calculada; no es necesariamente toda la tabla interna del algoritmo.

Utilización Fracción de capacidad de servicio ocupada en promedio.

Valor esperado Promedio ponderado por probabilidades; no garantiza ese pago en una realización particular.

Valor de información Mejora esperada al permitir decisiones basadas en evidencia; EVSI corresponde a información muestral y EVPI a información perfecta.

Variable de decisión Magnitud controlable que el decisor elige en el modelo.

Verosimilitud Probabilidad de observar una evidencia condicionada a un estado; no es la probabilidad posterior del estado.

Bibliografía

[Ahuja et al., 1993] Ahuja, R. K., Magnanti, T. L., and Orlin, J. B. (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall.

[Bertsimas and Tsitsiklis, 1997] Bertsimas, D. and Tsitsiklis, J. N. (1997). *Introduction to Linear Optimization*. Athena Scientific.

[Dantzig, 1963] Dantzig, G. B. (1963). *Linear Programming and Extensions*. Princeton University Press.

[Gross et al., 2008] Gross, D., Shortle, J. F., Thompson, J. M., and Harris, C. M. (2008). *Fundamentals of Queueing Theory*. Wiley, 4 edition.

[Harris, 1913] Harris, F. W. (1913). How many parts to make at once. *Factory, The Magazine of Management*, 10:135–136.

[Hyndman and Athanasopoulos, 2021] Hyndman, R. J. and Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*. OTexts, 3 edition.

[Law, 2015] Law, A. M. (2015). *Simulation Modeling and Analysis*. McGraw-Hill Education, 5 edition.

[Montgomery, 2020] Montgomery, D. C. (2020). *Introduction to Statistical Quality Control*. Wiley, 8 edition.

[Nemhauser and Wolsey, 1988] Nemhauser, G. L. and Wolsey, L. A. (1988). *Integer and Combinatorial Optimization*. Wiley.

[Pinedo, 2016] Pinedo, M. L. (2016). *Scheduling: Theory, Algorithms, and Systems*. Springer, 5 edition.

[Ross, 2019] Ross, S. M. (2019). *Introduction to Probability Models*. Academic Press, 12 edition.

[Silver et al., 1998] Silver, E. A., Pyke, D. F., and Peterson, R. (1998). *Inventory Management and Production Planning and Scheduling*. Wiley, 3 edition.

[Wolsey, 1998] Wolsey, L. A. (1998). *Integer Programming*. Wiley.